軟硬體專題設計文件書

水果(鳳梨)電子鼻非破壞性熟度檢測之系統開發計畫

課程名稱：軟硬體專題課程(2)

系級：資訊工程學系

成員：

B1229062 林冠妤

B1229066 陳怡禎

B1229067 廖文歆

B1144143 陳玟妤

指導老師：張哲維 張哲維 2026/04/11

繳交日期：2026 年 04 月 20 日

目錄

1. 系統簡介 3
1.1 文件目的 3
1.2 規格範圍 3
1.3 參考文件 3
2. 系統概述 4
2.1 系統目標 4
2.2 系統範圍 4
2.3 系統架構 7
2.3.1 整體系統架構圖 7
2.3.2 硬體架構設計 12
2.3.3 軟體架構設計 13
2.3.4 資料流程架構 15
2.4 軟/硬體建構項目需求概述 17
2.4.1 硬體建構需求 17
2.4.2 軟體建構需求 18
2.5 軟/硬體環境 18
2.5.1 開發環境 18
2.5.2 部署環境 19
2.5.3 使用者操作環境 20
2.6 一般限制 21
2.6.1 硬體限制 22
2.6.2 軟體與演算法限制 22
2.6.3 應用場景限制 23
3. 設計內容 24
3.1 軟/硬體建構項目架構 24
3.1.1 硬體建構架構 24
3.1.2 軟體建構架構 26
3.2 軟硬體組件設計說明 28
3.2.1 硬體組件詳細設計 28
3.2.2 軟體組件詳細設計 33
3.3 介面設計說明 37

3.3.1 介面結構概述 37
3.3.2 各頁面模組設計 37
3.3.3 UI 設計原則 41
3.3.4 雙語設計 41
3.3.5 Flask 模擬介面與驗證流程 41
3.4 作業程序設計說明 43
3.4.1 系統部署作業流程 43
3.4.2 系統校正作業流程(空氣基線校正) 46
3.4.4 系統維護作業流程 50
3.4.5 End-to-End 完整作業流程 51
3.5 輸入/輸出設計說明 52
3.5.1 共用輸入設計 52
3.5.2 共用輸出設計 53
3.5.3 消費端 App 輸入/輸出設計 53
3.5.4 農民端 App 輸入/輸出設計 54
3.6 其他設計說明 54
3.6.2 系統狀態機（State Machine） 56
3.6.3 錯誤處理設計 58
3.6.4 音效與語音提示設計 60
3.6.5 可擴充性設計 60
4. 需求追溯與版本管理 61
4.1 需求追溯說明 61
4.2 程式版本管理 61
4.3 分支策略 62
4.5 文件版本控管 62
附錄 63

1. 系統簡介

本系統為「鳳梨成熟度辨識系統」，結合電子鼻氣體感測器陣列、Arduino Mega、Raspberry Pi3 以及機器學習模型，用於非破壞性判別鳳梨之成熟階段。系統透過量測鳳梨周圍揮發性氣體(VOC)訊號，並經由特徵抽取與分類模型推論，輸出四階段成熟度結果，以協助農民、驗收人員以及消費者進行快速的成熟判定。

1.1 文件目的

本設計規格書之目的如下:

- 明確定義鳳梨成熟度辨識系統之功能、效能及介面需求，作為開發與測試之依據。
- 說明系統所需之軟體與硬體架構，以利開發人員進行系統設計與實作。
- 作為專題各模組(感測端、模型端、前端介面)溝通協調之共同技術文件，降低實作過程之誤解。

1.2 規格範圍

本規格書範圍涵蓋下列內容:

- 鳳梨成熟度四階段(Stage 0~Stage 3)之量測、特徵計算與分類推論流程之規格定義。
- 感測端(氣體感測器陣列 + Arduino Mega)、運算端(Raspberry Pi 3 上之機器學習模型-ExtraTrees 與推論程式)、使用者端(APP 介面/網頁介面/終端輸出)之功能與介面需求。
- 系統運作流程、資料格式與主要效能指標(如準確率、推論時間範圍等)之描述。

本規格書不包含:

- 鳳梨栽培管理、田間採樣標準操作流程(SOP)之詳細農業作業規範。
- 非本專題範圍之其他作物或其他感測硬體之規格。

1.3 參考文件

本系統設計與撰寫本規格書時，主要參考下列文件與資料:

- 「鳳梨成熟度辨識系統」專題例會會議之簡報與進度報告(含感測設計、特徵工程、模型訓練與部署結果)。
- 軟體工程課程提供之「設計文件書(參照格式)」範例，作為章節架構與撰寫格式之依據。
- 相關機器學習與電子鼻(Electronic Nose)應用之研究文獻，作為特徵選取與分類模型設計之參考。

- 開發工具與平台之官方文件，例如 Arduino、Raspberry Pi 3、Python 套件(如 scikit-learn、Flask 等)之技術說明文件。

2. 系統概述

本章說明本系統之整體架構、設計方法、作業流程、軟硬體需求、運行環境與一般限制，作為後續詳細設計之基礎。

2.1 系統目標

本系統採原型導向開發方式，先建置資料蒐集與模型訓練環境，再逐步整合為部署原型系統,其主要設計方法及工具如下:

- 資料蒐集與標註
  - 使用氣體感測器陣列量測鳳梨與空氣樣本之 VOC 訊號，產出 per-day features 原始資料表。
  - 依外觀與實際成熟狀態，標註四階段成熟度標籤(Stage 0~Stage 3)。
- 模型訓練方式
  - 訓練階段採 Teacher-Student 架構:以 CatBoost 為 Teacher Model，輔助標註與覆蓋率改善;以 ExtraTrees 為 Student Model 作為最終部署模型。\
  - 透過特徵重要度分析(MI Top Features)選出 11 維部署特徵，並以 30 秒訊號窗口作為推論單位。
- 部署與應用工具
  - 在 Raspberry Pi 3 上以 Python 執行 ExtraTrees 部署模型，搭配 inference_30s.py 進行 30 秒窗口特徵計算與推論。
  - 使用 Flask 建立本機網頁介面模擬，並以 SSH/Paramiko 遠端控制 Raspberry Pi 校正與推論流程。

2.2 系統範圍

本節以流程方式描述系統自啟動、校正、量測至輸出成熟度結果之作業程序，對應後續第 3 章之詳細設計。

可分為下列主要流程(搭配以下系統流程圖):

1. 系統啟動與環境檢查
   - 啟動 Raspberry Pi 及 Arduino，載入 airbaseline 與部署模型檔案。
2. 校正(空氣基線取得)
   - 於無鳳梨情況下進行多次空氣量測，計算各感測器 30 秒窗口之 baseline。
3. 量測與特徵計算

- 將待測鳳梨置於量測環境,連續讀取感測器訊號，並依 30 秒窗口計算 11 維特徵。

4. 模型推論與後處理
   - 以 ExtraTrees 模型輸出四階段成熟度預測,並套用 Guard/Override 後處理規則。
5. 結果呈現與紀錄
   - 透過 Flask 模擬介面顯示推論結果,並可視需求將結果與原始特徵資料記錄於檔案。

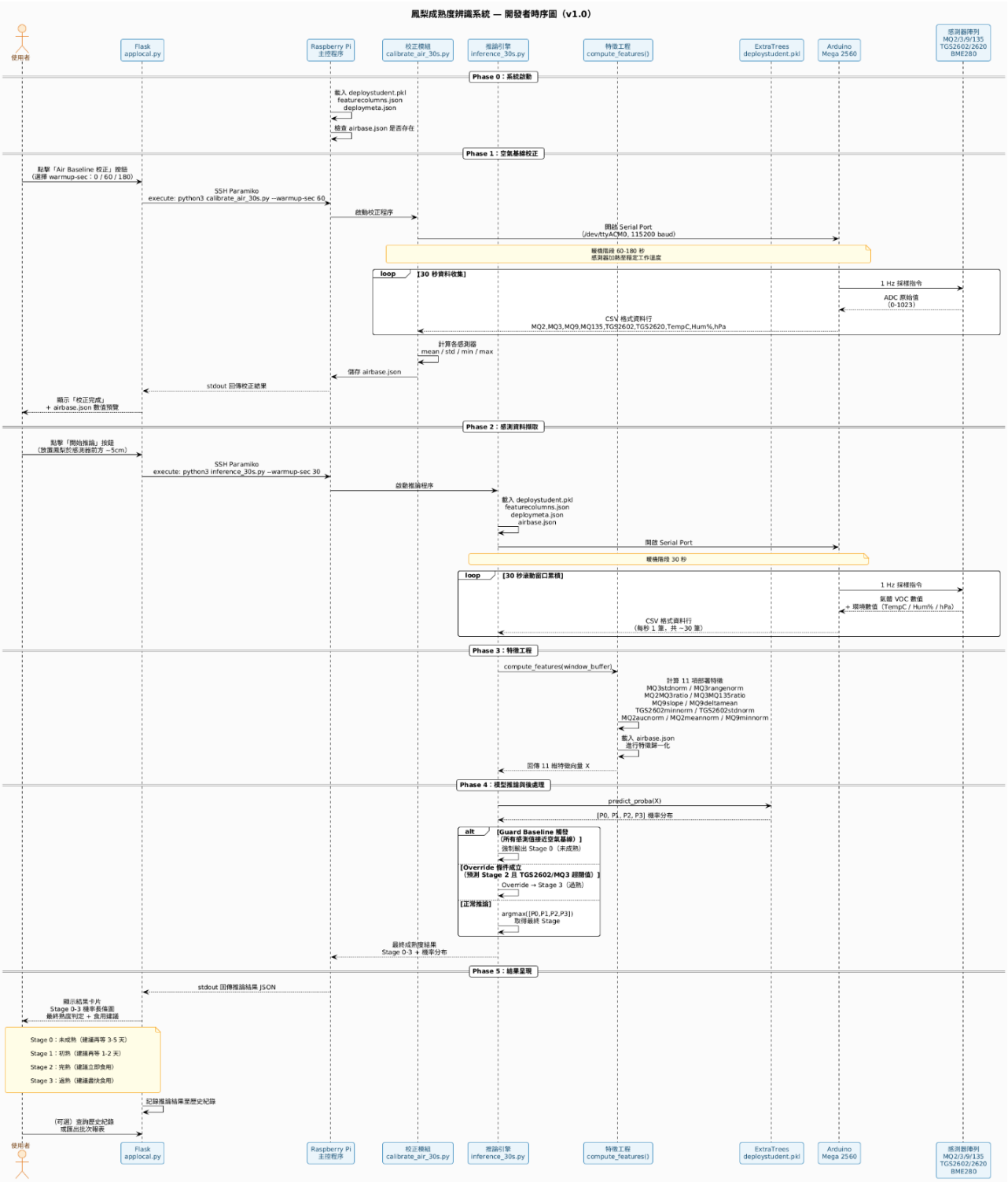

鳳梨成熟度辨識系統 — 開發者時序圖（v1.0）
使用者
Flask
applocal.py
Raspberry Pi
主控程序
校正模組
calibrate_air_30s.py
推論引擎
inference_30s.py
特徵工程
compute_features()
ExtraTrees
deploystudent.pkl
Arduino
Mega 2560
感測器陣列
MQ2/3/9/135
TGS2602/2620
BME280
Phase 0：系統啟動
載入 deploystudent.pkl
featurecolumns.json
deploymeta.json
檢查 airbase.json 是否存在
Phase 1：空氣基線校正
點擊「Air Baseline 校正」按鈕
（選擇 warmup-sec：0 / 60 / 180）
SSH Paramiko
execute: python3 calibrate_air_30s.py --warmup-sec 60
啟動校正程序
開啟 Serial Port
（/dev/ttyACM0, 115200 baud）
暖機階段 60-180 秒
感測器加熱至穩定工作溫度
loop
[30 秒資料收集]
1 Hz 採樣指令
ADC 原始值
（0-1023）
CSV 格式資料行
MQ2,MQ3,MQ9,MQ135,TGS2602,TGS2620,TempC,Hum%,hPa
計算各感測器
mean / std / min / max
儲存 airbase.json
stdout 回傳校正結果
顯示「校正完成」
+ airbase.json 數值預覽
Phase 2：感測資料擷取
點擊「開始推論」按鈕
（放置鳳梨於感測器前方 ~5cm）
SSH Paramiko
execute: python3 inference_30s.py --warmup-sec 30
啟動推論程序
載入 deploystudent.pkl
featurecolumns.json
deploymeta.json
airbase.json
開啟 Serial Port
暖機階段 30 秒
loop
[30 秒滾動窗口累積]
1 Hz 採樣指令
氣體 VOC 數值
+ 環境數值（TempC / Hum% / hPa）
CSV 格式資料行
（每秒 1 筆，共 ~30 筆）
Phase 3：特徵工程
compute_features(window_buffer)
計算 11 項部署特徵
MQ3stdnorm / MQ3rangenorm
MQ2MQ3ratio / MQ3MQ135ratio
MQ9slope / MQ9deltamean
TGS2602minnorm / TGS2602stdnorm
MQ2aucnorm / MQ2meannorm / MQ9minnorm
載入 airbase.json
進行特徵歸一化
回傳 11 維特徵向量 X
Phase 4：模型推論與後處理
predict_proba(X)
[P0, P1, P2, P3] 機率分布
alt
[Guard Baseline 觸發
（所有感測值接近空氣基線）]
強制輸出 Stage 0（未成熟）
[Override 條件成立
（預測 Stage 2 且 TGS2602/MQ3 超閾值）]
Override → Stage 3（過熟）
[正常推論]
argmax([P0,P1,P2,P3])
取得最終 Stage
最終成熟度結果
Stage 0-3 + 機率分布
Phase 5：結果呈現
stdout 回傳推論結果 JSON
顯示結果卡片
Stage 0-3 機率長條圖
最終熟度判定 + 食用建議
Stage 0：未成熟（建議再等 3-5 天）
Stage 1：初熟（建議再等 1-2 天）
Stage 2：完熟（建議立即食用）
Stage 3：過熟（建議盡快食用）
紀錄推論結果至歷史紀錄
（可選）查詢歷史紀錄
或匯出批次報表

圖 2.2. 1-1 系統流程圖

2.3 系統架構

本系統採用嵌入式機器學習架構，結合硬體感測與邊緣運算技術,實現鳳梨成熟度的即時非破壞性檢測。系統架構分為三大層次：感測層、運算處理層與應用介面層，以支援從 VOC 量測到成熟度判定之完整流程。

2.3.1 整體系統架構圖

系統採用分層式架構設計,各層功能如下:

架構層次	功能描述
感測層 (Sensing Layer)	透過 Arduino 整合多種氣體感測器(MQ2, MQ3, MQ9, MQ135, TGS2602, TGS2620)及環境感測器（BME280：溫度、濕度、氣壓,擷取鳳梨揮發性有機化合物(VOC)特徵數據，並將資料透過序列埠傳送至 Raspberry Pi 3。
運算處理層 (Processing Layer)	由 Raspberry Pi 3 執行特徵前處理、11 維特徵計算、機器學習(ExtraTrees)模型推論，進行 30 秒即時分析，輸出成熟度分級結果(Stage 0-3)，校正與 Guard/Override 後處理邏輯。
應用介面層 (Application Layer)	以 Flask 建立本機 Web 介面,整合 Air Baseline 校正按鈕、推論啟動按鈕與結果顯示區塊,並支援透過 SSH 從外部主機觸發校正與推論指令

Table 1: 系統架構層次說明

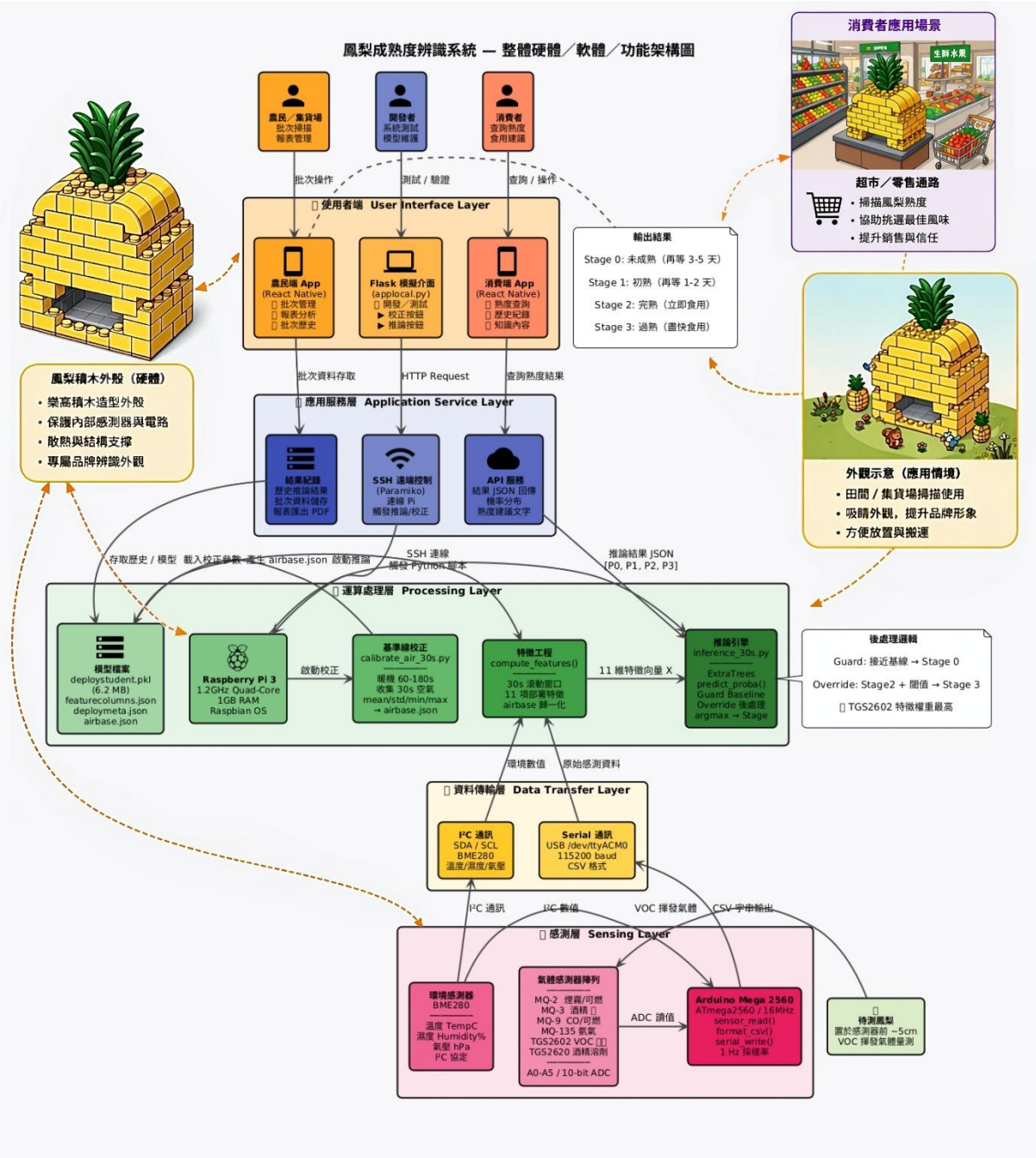

鳳梨成熟度辨識系統 — 整體硬體／軟體／功能架構圖
農民／集貨場
批次掃描
報表管理
開發者
系統測試
模型維護
消費者
查詢熟度
食用建議
批次操作
測試 / 驗證
查詢 / 操作
使用者端 User Interface Layer
農民端 App
(React Native)
批次管理
報表分析
批次歷史
Flask 模擬介面
(applocal.py)
開發／測試
校正按鈕
推論按鈕
消費端 App
(React Native)
熟度查詢
歷史紀錄
知識內容
輸出結果
Stage 0: 未成熟（再等 3-5 天）
Stage 1: 初熟（再等 1-2 天）
Stage 2: 完熟（立即食用）
Stage 3: 過熟（盡快食用）
消費者應用場景
生鮮水果
超市／零售通路
• 掃描鳳梨熟度
• 協助挑選最佳風味
• 提升銷售與信任
鳳梨積木外殼（硬體）
• 樂高積木造型外殼
• 保護內部感測器與電路
• 散熱與結構支撐
• 專屬品牌辨識外觀
批次資料存取
HTTP Request
查詢熟度結果
應用服務層 Application Service Layer
結果紀錄
歷史推論結果
批次資料儲存
報表匯出 PDF
SSH 遠端控制
(Paramiko)
連線 Pi
觸發推論/校正
API 服務
結果 JSON 回傳
機率分布
熟度建議文字
外觀示意（應用情境）
• 田間 / 集貨場掃描使用
• 吸睛外觀，提升品牌形象
• 方便放置與搬運
存取歷史 / 模型
載入校正參數
產生 airbase.json
啟動推論
SSH 連線
觸發 Python 腳本
推論結果 JSON
[P0, P1, P2, P3]
運算處理層 Processing Layer
模型檔案
deploystudent.pkl
(6.2 MB)
featurecolumns.json
deploymeta.json
airbase.json
Raspberry Pi 3
1.2GHz Quad-Core
1GB RAM
Raspbian OS
啟動校正
基準線校正
calibrate_air_30s.py
暖機 60-180s
收集 30s 空氣
mean/std/min/max
→ airbase.json
特徵工程
compute_features()
30s 滾動窗口
11 項部署特徵
airbase 歸一化
11 維特徵向量 X
推論引擎
inference_30s.py
ExtraTrees
predict_proba()
Guard Baseline
Override 後處理
argmax → Stage
後處理邏輯
Guard: 接近基線 → Stage 0
Override: Stage2 + 閾值 → Stage 3
TGS2602 特徵權重最高
環境數值
原始感測資料
資料傳輸層 Data Transfer Layer
I²C 通訊
SDA / SCL
BME280
溫度/濕度/氣壓
Serial 通訊
USB /dev/ttyACM0
115200 baud
CSV 格式
I²C 通訊
I²C 數值
VOC 揮發氣體
CSV 字串輸出
感測層 Sensing Layer
環境感測器
BME280
溫度 TempC
濕度 Humidity%
氣壓 hPa
I²C 協定
氣體感測器陣列
MQ-2 煙霧/可燃
MQ-3 酒精
MQ-9 CO/可燃
MQ-135 氨氣
TGS2602 VOC
TGS2620 酒精溶劑
A0-A5 / 10-bit ADC
ADC 讀值
Arduino Mega 2560
ATmega2560 / 16MHz
sensor_read()
format_csv()
serial_write()
1 Hz 採樣率
特測鳳梨
置於感測器前 ~5cm
VOC 揮發氣體量測

圖 2.3. 1.1 系統架構圖

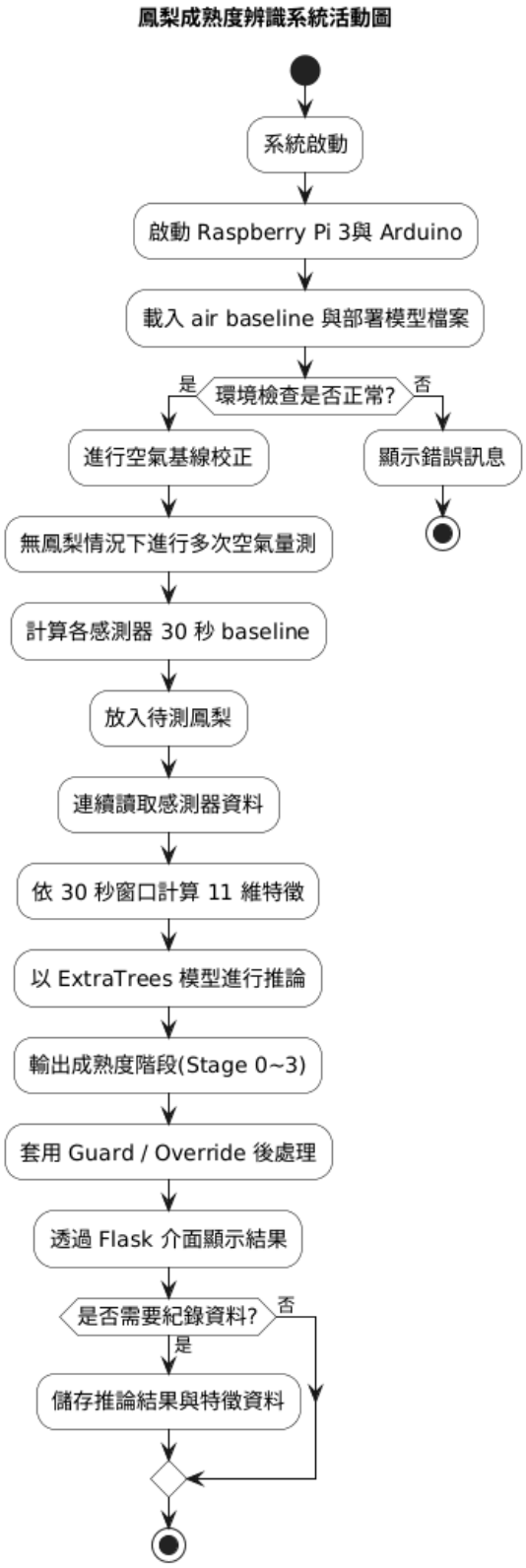

鳳梨成熟度辨識系統活動圖
系統啟動
啟動 Raspberry Pi 3與 Arduino
載入 air baseline 與部署模型檔案
環境檢查是否正常?
是
否
進行空氣基線校正
顯示錯誤訊息
無鳳梨情況下進行多次空氣量測
計算各感測器 30 秒 baseline
放入待測鳳梨
連續讀取感測器資料
依 30 秒窗口計算 11 維特徵
以 ExtraTrees 模型進行推論
輸出成熟度階段(Stage 0~3)
套用 Guard / Override 後處理
透過 Flask 介面顯示結果
是否需要紀錄資料?
否
是
儲存推論結果與特徵資料

圖 2.3. 1.2 系統活動圖

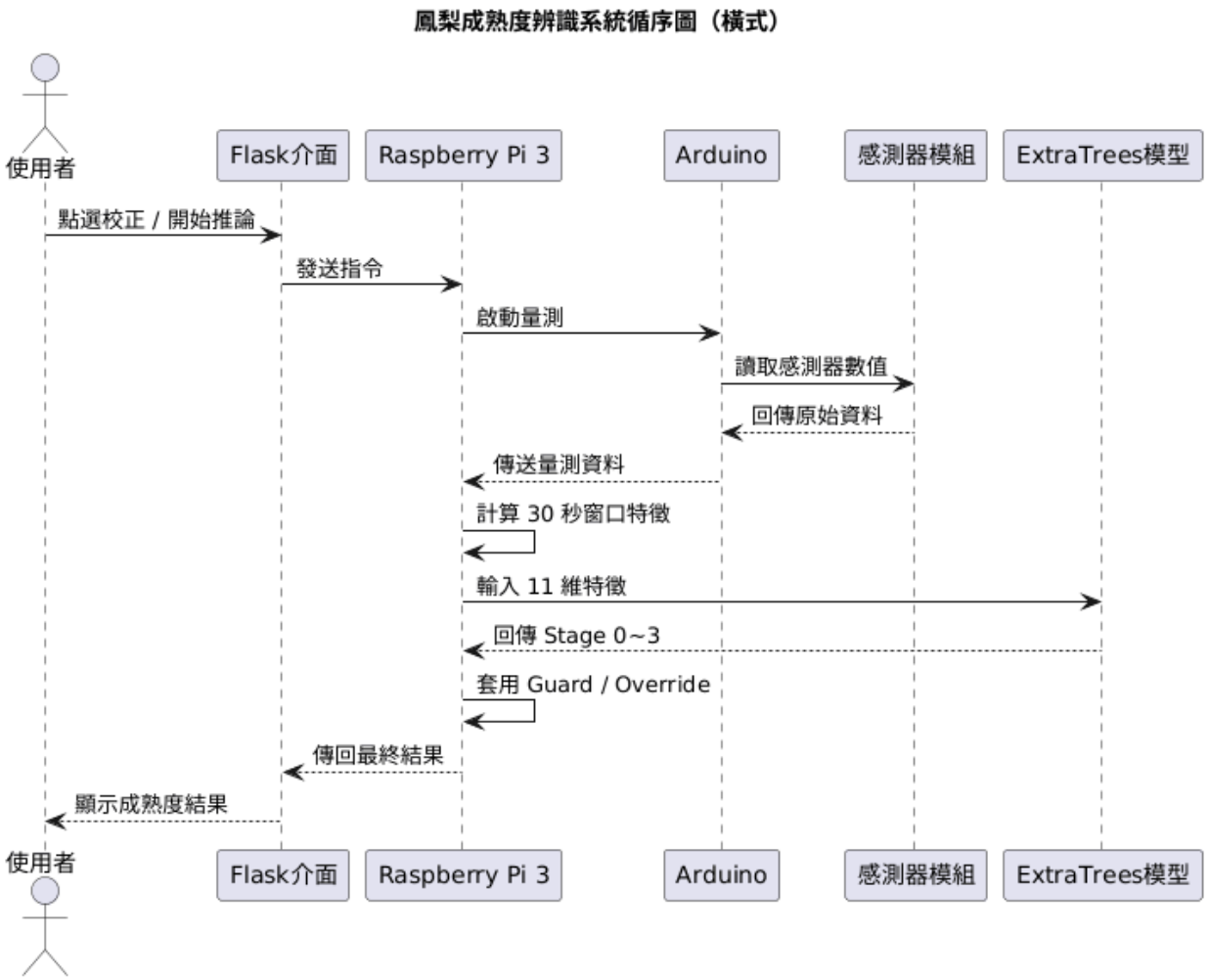

鳳梨成熟度辨識系統循序圖（橫式）
使用者
Flask介面
Raspberry Pi 3
Arduino
感測器模組
ExtraTrees模型
點選校正 / 開始推論
發送指令
啟動量測
讀取感測器數值
回傳原始資料
傳送量測資料
計算 30 秒窗口特徵
輸入 11 維特徵
回傳 Stage 0~3
套用 Guard / Override
傳回最終結果
顯示成熟度結果
使用者
Flask介面
Raspberry Pi 3
Arduino
感測器模組
ExtraTrees模型

圖 2.3. 1.3 系統循序圖

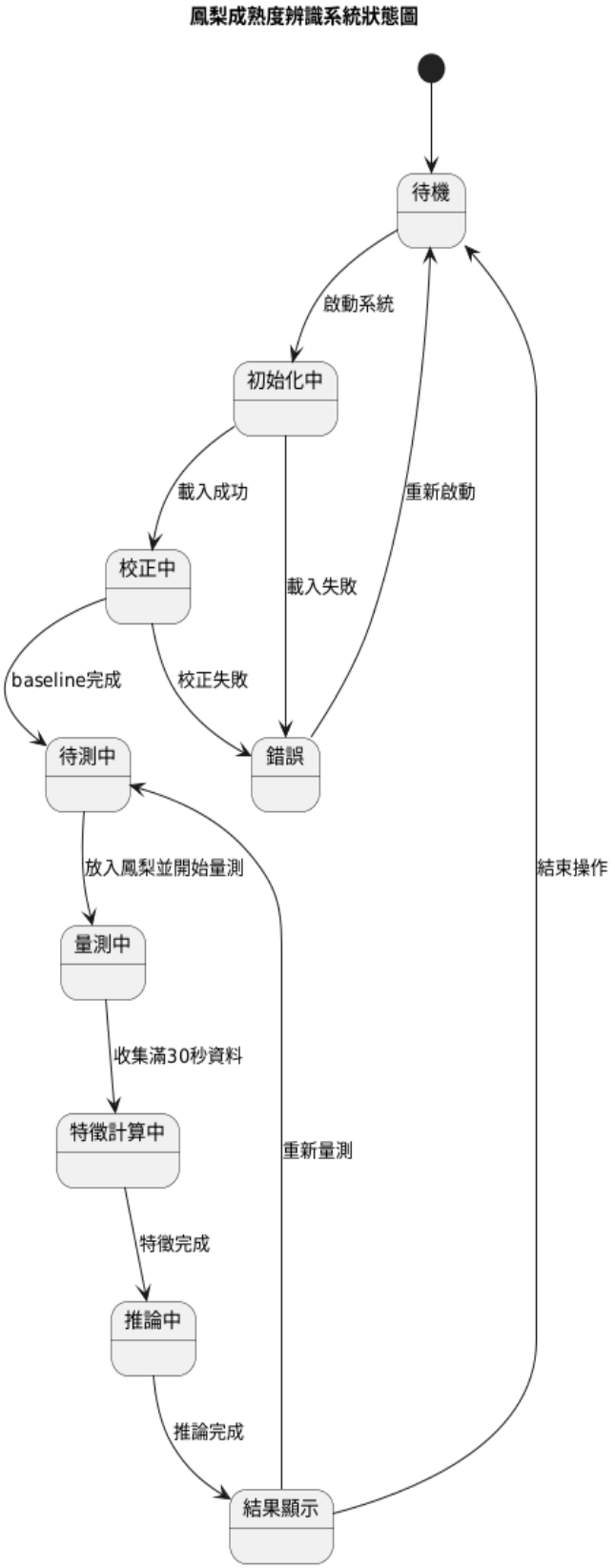

鳳梨成熟度辨識系統狀態圖
待機
啟動系統
初始化中
載入成功
重新啟動
校正中
載入失敗
baseline完成
校正失敗
待測中
錯誤
放入鳳梨並開始量測
結束操作
量測中
收集滿30秒資料
特徵計算中
重新量測
特徵完成
推論中
推論完成
結果顯示

圖 2.3. 1.4 系統狀態圖

2.3.2 硬體架構設計

硬體架構以 Arduino Mega 2560 作為感測器資料擷取單元,Raspberry Pi 3 作為邊緣運算核心。

感測器陣列配置:

- 氣體感測器: MQ2(煙霧/可燃氣體)、MQ3(酒精)、MQ9(CO/可燃氣體)、MQ135(空氣品質/氨氣)、TGS2602(VOC/氨氣)、TGS2620(酒精/有機溶劑)
- 環境感測器: BME280（溫度、濕度、氣壓三合一）
- 資料傳輸: Arduino 透過 Serial Port(串列埠)以 CSV 格式傳送即時感測數據至 Raspberry Pi

邊緣運算單元:

- 硬體平台: Raspberry Pi 3 Model B
- 作業系統: Raspbian Linux
- 運算負載: ExtraTrees 機器學習模型(模型大小: 6.2MB, 特徵維度: 11)

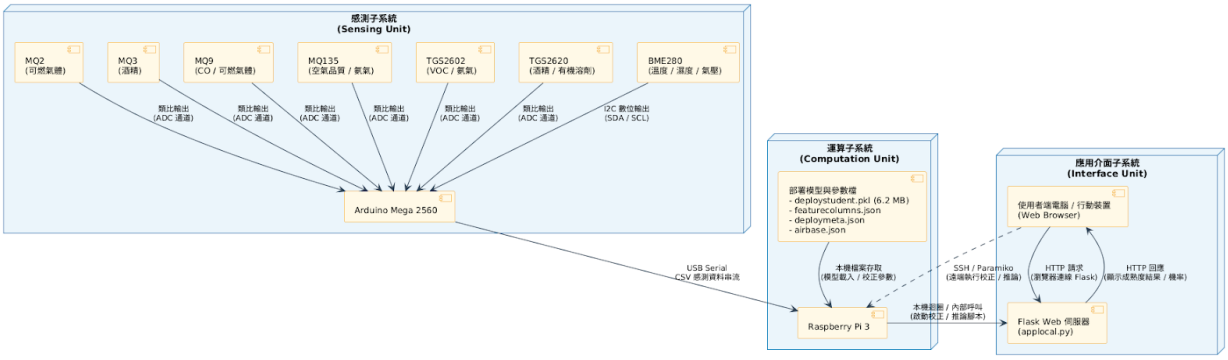


圖 2.3. 1.5 硬體連接架構圖

2.3.3 軟體架構設計

軟體架構採用模組化設計,包含數據前處理、特徵工程、模型推論與結果校正四大模組。

軟體模組	功能說明
數據擷取模組	Arduino 感測器數據讀取、CSV 格式輸出、Serial 通訊協定處理
特徵工程模組	計算 11 項部署特徵(MQ3stdnorm, MQ2MQ3ratio, TGS2602minnorm 等)、氣體濃度比值與變化率分析
基準線校正模組 (calibrate_air_30s.py)	執行 30 秒空氣基準線量測、產生 airbase.json 參數檔、提供歸一化參考值
模型推論模組 (inference_30s.py)	載入 ExtraTrees 模型(deploystudent.pkl)、執行 30 秒窗口推論、應用 Guard Baseline 與 Override 邏輯
結果校正模組	依 PID(鳳梨個體 ID)執行後處理、提升樣本準確率至 80.49%
網頁應用模組 (applocal.py)	Flask 框架、SSH Paramiko 遠端控制、即時顯示機率分布與預測結果

Table 2: 軟體模組功能說明

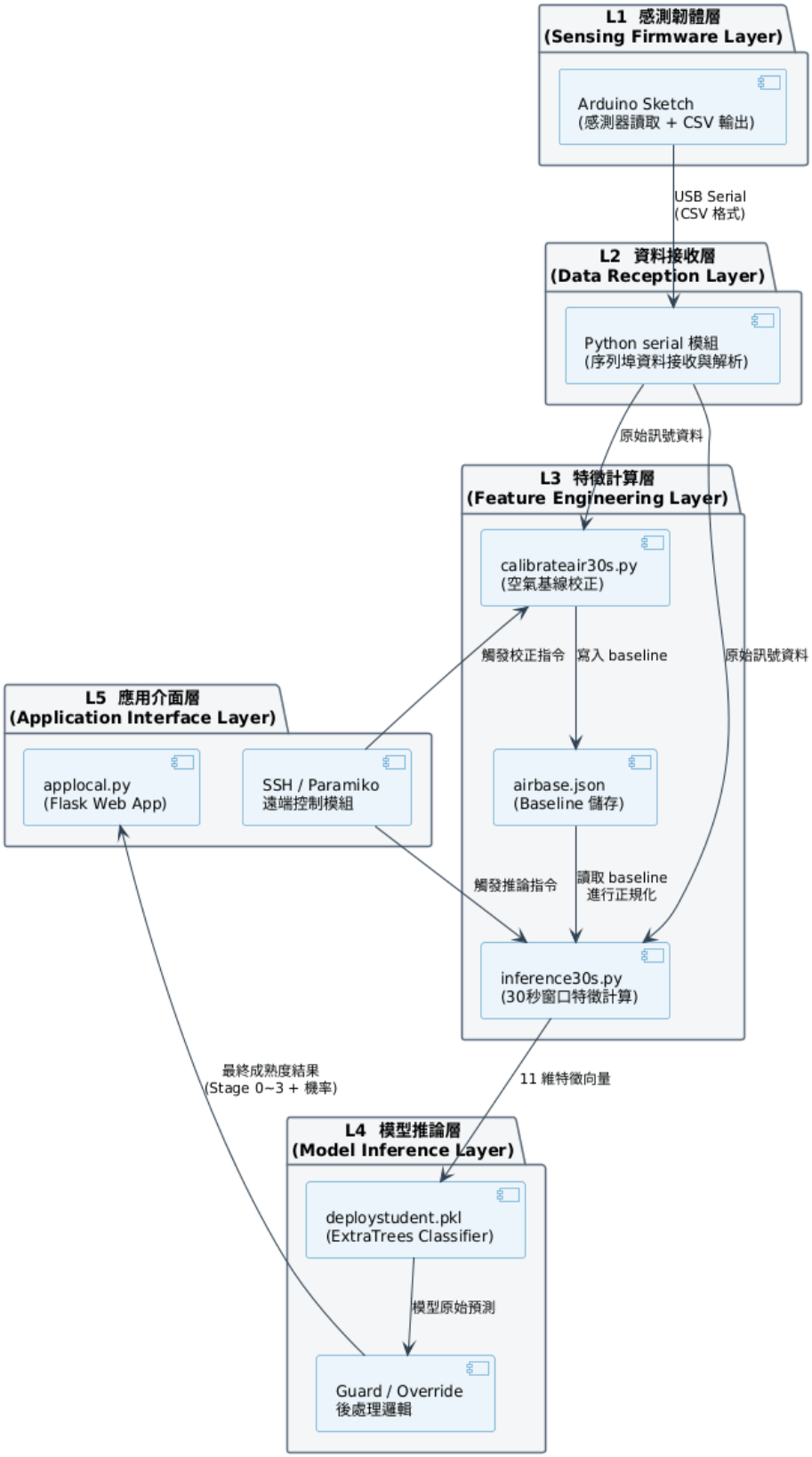


圖 2.3. 1.6 軟體分層架構圖

2.3.4 資料流程架構

系統資料處理流程分為訓練階段與部署階段:

訓練階段資料流:

1. 收集 14 顆鳳梨、333 筆每日 VOC 量測數據
2. 特徵工程: 計算 159 項 PID-specific 特徵 + 7 項 shared-by-date 特徵
3. 標記 4 階段成熟度(Stage 0: 32, Stage 1: 25, Stage 2: 29, Stage 3: 21)
4. Stage 3 覆蓋率提升: 透過 Pseudo Labeling 將覆蓋率從 57.14%提升至 69 筆
5. 訓練 Teacher Model(CatBoost) → 知識蒸餾至 Student Model(ExtraTrees)
6. 模型壓縮: 從完整特徵集縮減至 11 項核心特徵,模型大小 6.2 MB

部署階段資料流:

1. Arduino 每秒讀取 8 維感測器數據(6 種氣體 + 溫度 + 濕度)
2. 累積 30 秒數據窗口
3. 計算 11 項特徵(dayratio 除外,因無日期資訊)
4. 載入 airbase.json 進行歸一化
5. ExtraTrees 模型推論,輸出 4 階段機率分布
6. 應用 Guard Baseline(基準線保護)與 Override 邏輯(針對 Stage 2-3 的 TGS2602/MQ3 特徵)
7. 輸出最終成熟度預測(argmax)

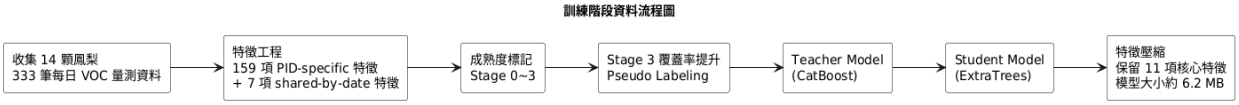


圖 2.3. 1.7 訓練流程圖

圖 2.3. 1.8 部屬階段資料流程圖

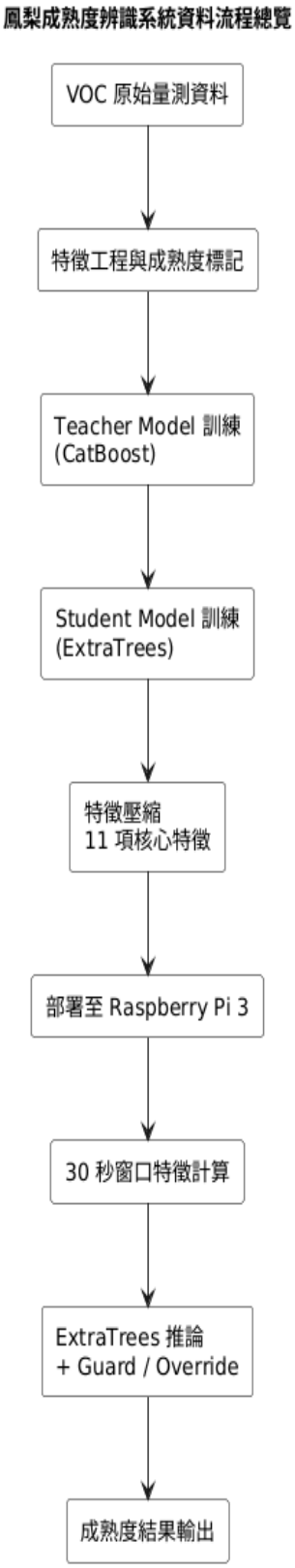


圖 2.3. 1.9 訓練與部屬整合資料流程圖

2.4 軟/硬體建構項目需求概述

本節彙整系統實作所需之主要硬體與軟體建構項目,作為採購與建置之依據。

2.4.1 硬體建構需求

項目	規格	數量	用途說明
Arduino Mega 2560	ATmega2560, 16MHz	1	感測器資料擷取與 Serial 輸出
Raspberry Pi 3 Model B	1.2GHz Quad-Core, 1GB RAM	1	邊緣運算、模型推論
MQ-2 氣體感測器	煙霧/可燃氣體檢測	1	VOC 特徵擷取
MQ-3 氣體感測器	酒精檢測	1	酒精類揮發物檢測
MQ-9 氣體感測器	CO/可燃氣體檢測	1	一氧化碳類特徵
MQ-135 氣體感測器	空氣品質/氨氣檢測	1	氨氣類揮發物
TGS2602 感測器	VOC/氨氣高靈敏度檢測	1	關鍵 VOC 特徵(模型中權重最高)
TGS2620 感測器	酒精/有機溶劑檢測	1	有機溶劑類特徵
BME280	溫度、濕度、氣壓感測器	1	溫度、濕度、大氣壓力監測
麵包板	標準尺寸	1	電路連接
杜邦線	公對公/公對母	若干	訊號連接
USB 連接線	Type A to Type B	1	Arduino 與 Raspberry Pi 連接
電源供應器	5V 2.5A	1	Raspberry Pi 供電

Table 3: 硬體建構項目清單

2.4.2 軟體建構需求

軟體項目	版本	用途說明
Raspbian OS	最新穩定版	Raspberry Pi 作業系統
Python	3.7+	主要開發語言
scikit-learn	最新版	ExtraTrees 模型支援
NumPy	最新版	數值運算
Pandas	最新版	數據處理
Flask	2.0+	網頁應用框架
Paramiko	最新版	SSH 遠端控制
PySerial	最新版	Serial Port 通訊
Arduino IDE	1.8+	Arduino 韌體開發

Table 4: 軟體建構項目清單

核心部署檔案:

- deploystudent.pkl: ExtraTrees 模型權重檔(6.2 MB)
- featurecolumns.json: 11 項特徵名稱清單
- deploymeta.json: 模型元資料
- airbase.json: 空氣基準線參數檔
- inference_30s.py: 推論主程式
- calibrate_air_30s.py: 校正主程式
- applocal.py: Flask 網頁應用

2.5 軟/硬體環境

2.5.1 開發環境

環境類別	規格說明
模型訓練環境	Jupyter Notebook、Python 3.8+、CatBoost(Teacher)、scikit-learn ExtraTrees(Student)
Arduino 開發環境	Arduino IDE 1.8+、C/C++程式語言、Serial 通訊協定開發
Raspberry Pi 開發環境	SSH 遠端連線、Python 3.7+、Flask 開發伺服器
版本控制	Git、GitHub repository

Table 5: 開發環境規格

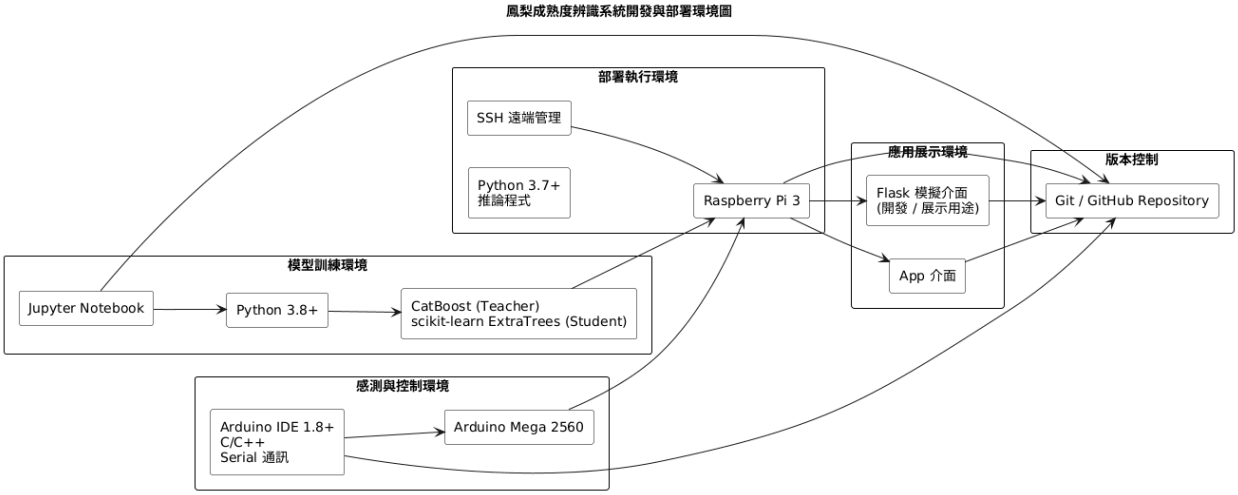


圖 2.5. 1.1 軟/硬體開發與部署環境圖

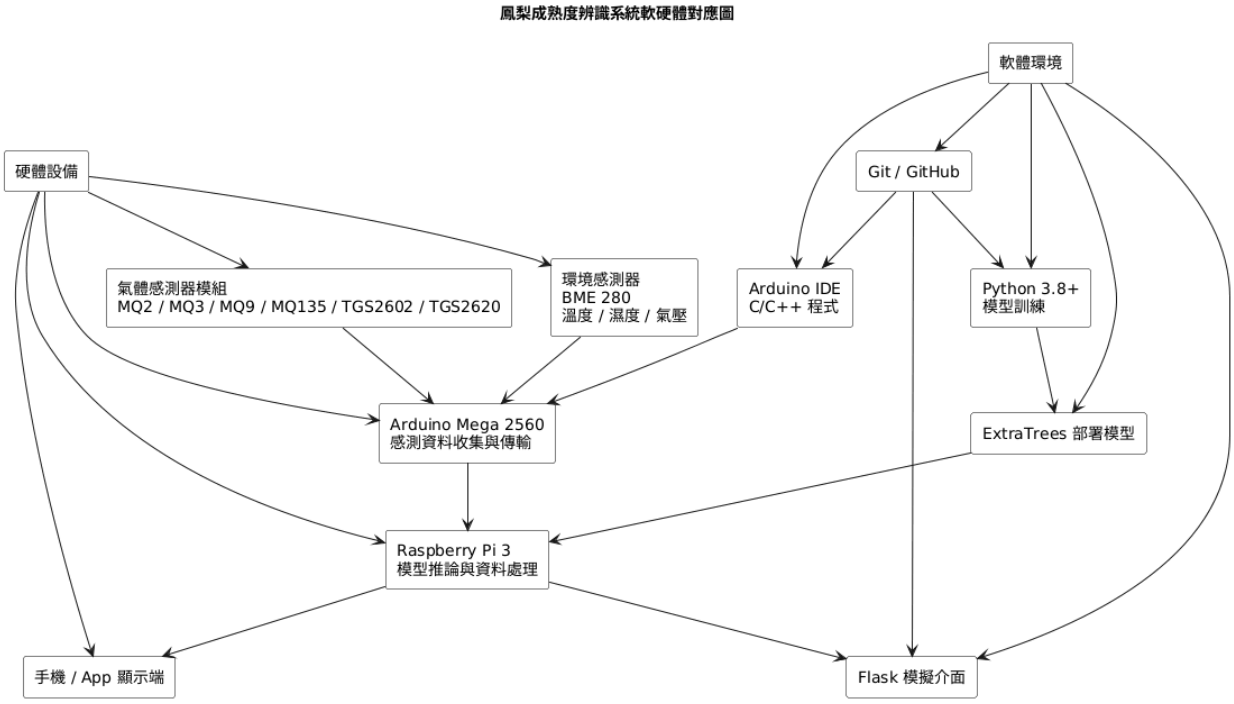


圖 2.5. 1.2 軟硬體對應圖

2.5.2 部署環境

環境類別	規格說明
硬體平台	Raspberry Pi 3 Model B（1.2GHz ARM Cortex-A53，1GB RAM）
感測與控制平台	Arduino Mega 2560，負責整合氣體感測器與環境感測器資料，並透過 Serial 傳送至 Raspberry Pi
作業系統	Raspberry Pi OS（Debian-based Linux）

Python 運行環境	Python 3.7+、虛擬環境(venv)
網路連線	透過 Wi-Fi 或乙太網路進行 Raspberry Pi 遠端連線與系統管理；App 為正式展示介面，Flask 僅作為開發與模擬測試用途
Serial 通訊	Arduino Mega 2560 與 Raspberry Pi 3 以 USB Serial 連接，裝置路徑為 /dev/ttyACM0（實際依系統辨識結果而定）
儲存空間需求	至少 500 MB 可用空間(包含作業系統、Python 函式庫、模型檔案)
部屬模型	ExtraTrees 分類模型，部署版使用 11 項核心特徵進行 30 秒窗口成熟度推論
介面與顯示	系統結果可透過 App 顯示；Flask 介面主要用於開發階段之功能模擬、測試與展示

Table 6: 部署環境規格

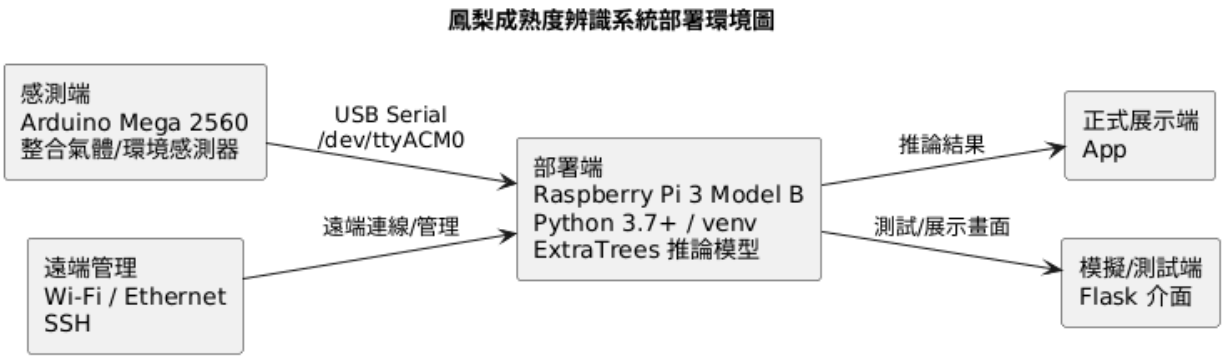


圖 2.5. 1.3 部屬環境架構圖

2.5.3 使用者操作環境

本系統正式使用情境以 App 作為主要操作與結果展示介面，使用者可透過 App 查看鳳梨成熟度辨識結果與相關系統回饋資訊。

另外於開發、測試與展示階段，系統提供 Flask 模擬介面，用以驗證校正、推論與結果顯示功能，主要由開發／測試人員使用。

使用者操作環境說明如下：

- 正式操作介面：App
- 模擬／測試介面：Flask Web 介面
- 網路需求：使用者端裝置需能與 Raspberry Pi 所在系統連線環境互通，以接收推論結果
- 顯示需求：App 介面可於一般行動裝置螢幕正常顯示；Flask 模擬介面建議於 1280 × 720 以上解析度使用，以利測試與展示

- 作業環境：
  - App：Android / ios 手機皆可或平板等行動裝置
  - Flask 模擬介面：Windows 10/11、macOS、Linux 等支援現代瀏覽器之作業系統
- 支援瀏覽器（僅 Flask 模擬介面）：Chrome、Firefox、Safari 或其他相容瀏覽器最新版

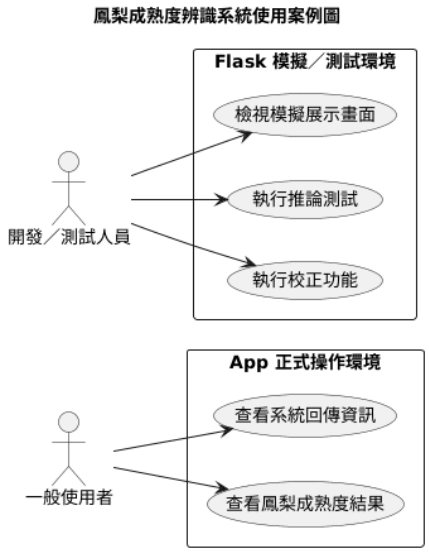


圖 2.5. 1.4 使用案例圖

2.6 一般限制

本系統雖已完成鳳梨成熟度辨識之感測、推論與結果展示流程，但在硬體資源、演算法設計、應用場景與系統可靠性等方面，仍存在若干限制，而本節限制項目彙整如下 Table 7 所示，後續 2.6.1～2.6.4 分別說明各類限制之細節。

限制類別	主要內容	可能影響
硬體限制	感測器需暖機、Raspberry Pi 3 運算資源有限、Serial 通訊需穩定	影響量測穩定性與推論效率
軟體與演算法限制	部署僅用 11 項特徵、固定 30 秒窗口、Stage 3 樣本較少	影響辨識準確率與即時性
應用場景限制	僅能進行非破壞性之單顆檢測、品種與儲存條件會影響結果	影響模型泛化能力
安全性與可靠性限制	Flask 僅供區域網路測試、尚無自動備份與自動復原機制	影響長期運行穩定性與安全性

Table 7: 系統一般限制總覽表

為使系統使用情境與結果解讀更為明確，本節彙整目前已知之限制條件如下。

2.6.1 硬體限制

1. 感測器暖機時間限制

氣體感測器啟動後需經過一定暖機時間方能達到穩定讀值，本系統建議暖機時間約為 60～180 秒，並可透過 warmup-sec 參數設定。若暖機時間不足，感測讀值可能產生較大波動，進而影響特徵計算與模型推論結果之穩定性。

2. 邊緣運算資源限制

本系統部署平台為 Raspberry Pi 3，其記憶體容量為 1 GB，處理器與記憶體資源有限，較不適合執行大型或高複雜度之深度學習模型。系統雖配備 32 GB SD 卡作為儲存媒介，但其主要用途為作業系統、程式與模型檔案存放，無法彌補運算資源不足之限制。故部署端採用較輕量之 ExtraTrees 分類模型，以兼顧推論效率與系統穩定性。

3. 感測器壽命限制

氣體感測器屬耗材型元件，使用壽命約為 2～5 年，長期使用後靈敏度可能衰退而影響量測準確度。系統需搭配定期校正、維護與必要之感測器更換作業，以維持長期量測品質。

4. 環境條件限制

感測器量測結果易受環境溫度與濕度影響，本系統建議操作環境維持於溫度 20～30˚C、相對濕度 40～70% 範圍內。若實際環境條件偏離上述範圍，可能增加雜訊與漂移，需透過額外校正或排程重新量測。

5. 序列通訊穩定性限制

Arduino Mega 2560 與 Raspberry Pi 3 之間以 USB Serial 傳輸資料，若兩端 baud rate 設定不一致或 Serial 線路不穩，將導致通訊異常或資料遺失。本系統建議使用 115200 baud rate，並確保 Serial Port 權限與接線狀態正確，以維持感測資料傳輸之穩定性。

2.6.2 軟體與演算法限制

1. 部署特徵受限

部署版本模型僅使用 11 項核心特徵進行推論，不包含 dayratio、dayindex 等需依賴日期資訊之特徵。相較於訓練階段使用之完整特徵集合，部署端可用資訊較少，因此推論表現可能略低於離線訓練結果。

2. 推論時間窗口限制

本系統固定採用 30 秒推論窗口，須累積完整 30 秒感測資料後方可進行一次成熟度判斷。此設計可提升短時間量測之穩定性，但無法提供毫秒級或秒級之即時連續追蹤能力。

3. 成熟度覆蓋率限制

在訓練資料中，Stage 3（最成熟／過熟階段）之樣本數相對不足，原始覆蓋率僅約57.14%，後續雖透過 Pseudo Labeling 擴增至 69 筆資料，但該階段之判斷能力仍可能受到樣本不平衡影響。

4. 模型辨識誤差限制

ExtraTrees Student Model 之 LOGO 交叉驗證平均準確率約為 86.15%，部署前樣本準確率約為 79.27%。此結果顯示系統仍存在一定比例之誤判，特別是在成熟度邊界接近之樣本上，較易出現 Stage 1／Stage 2、Stage 2／Stage 3 間之分類混淆。

5. 校正依賴性限制

本系統需先執行空氣基線校正程序，產生 airbase.json 檔案後，方能正確進行特徵歸一化與 Guard 判斷。若未完成或久未更新校正，系統對感測器輸入資料之解讀將產生偏移，可能導致推論失敗或結果偏差。

6. 個體差異限制

不同鳳梨個體在 VOC 釋放特性上存在差異，部分個體（PID）之辨識準確率較低，例如曾觀察到某 PID 約 58.33% 之辨識率。雖透過 Per-PID 校正與後處理規則可將整體樣本準確率提升至約 80.49%，但個別鳳梨之判定仍可能受個體差異影響。

2.6.3 應用場景限制

1. 非破壞性檢測限制

本系統採非破壞性檢測方式，主要根據鳳梨外部揮發性有機化合物（VOC）進行成熟度判斷，無法直接量測果肉內部之糖度、酸度或纖維質等品質指標。相關內部品質仍需搭配其他檢測方法或實務經驗綜合判斷。

2. 品種適用性限制

本系統訓練資料主要來自金鑽鳳梨品種與採樣條件，若應用於其他品種或顯著不同產區，可能因 VOC 釋放特性差異而影響辨識表現。必要時需重新蒐集資料並進行模型再訓練，方能確保於新情境下之準確度。

3. 儲存條件影響限制

鳳梨儲存溫度、放置時間與環境條件會改變 VOC 釋放模式，若實際應用之儲存條件與訓練資料蒐集時差異過大，可能降低模型辨識穩定度。建議於與訓練階段相近之環境條件下操作，或另行建立對應條件之模型。

4. 批次處理能力限制

目前系統設計以單顆鳳梨量測為主，尚未支援多顆鳳梨同時量測與自動分離判讀之機制。若需應用於大量批次檢測場域，仍須於硬體配置與演算法層面進行額外擴充。

5. 即時性限制

由於系統需等待完整 30 秒資料窗口後方能輸出結果，目前較適合作為短時間快速檢測工具，而非毫秒級或秒級之即時監測系統。

2.6.4 安全性與可靠性限制

1. 網路安全限制

目前 Flask 模擬介面主要設計用於區域網路環境下之開發、測試與展示，尚未實作 HTTPS 加密傳輸與完整身分驗證機制。因此不建議於公開網路環境中直接對外提供服務，未來如需正式上線須補強相關安全機制。

2. 資料備份限制

本系統尚未建立自動化資料備份機制，感測資料、推論結果與模型檔案之備份仍需依賴人工定期執行。若系統異常或儲存裝置損毀，可能造成部分資料遺失。

3. 異常復原能力限制

當 Serial 通訊中斷、感測器故障或部署程式異常終止時，系統目前缺乏自動重連、自動重啟與錯誤復原流程。故需由開發者或系統管理者手動排除故障並重新啟動相關服務，短時間內可能影響系統可用性。

4. 模型版本管理限制

部署端模型檔案目前尚未整合完整版本控管資訊與變更紀錄，未來若進行模型更新、特徵調整或部署環境修改，可能發生版本相容性問題。後續需補強模型版本編號、特徵欄位對應與部署歷程管理，以利追蹤與回溯。

3. 設計內容

3.1 軟/硬體建構項目架構

3.1.1 硬體建構架構

本系統硬體建構採用「感測陣列結合邊緣運算」之架構，整體可分為感測器陣列模組、資料擷取控制模組與邊緣運算模組三大單元，三大單元如下:

單元 1: 感測器陣列模組

- 氣體感測器子系統:
  - MQ 系列感測器(MQ2, MQ3, MQ9, MQ135): 類比訊號輸出,連接至 Arduino A0-A3 類比輸入腳位
  - TGS 系列感測器(TGS2602, TGS2620): 類比訊號輸出,連接至 Arduino A4-A5 類比輸入腳位
  - 各氣體感測器共用 5V 電源供應，並依模組需求進行訊號讀取
- 環境感測器子系統:

- BME280 氣壓感測器: 採 I2C 通訊協定，連接至 Arduino Mega 2560 之 SDA / SCL 腳位，用於量測環境溫度、濕度與氣壓資訊。

單元 2: 資料擷取控制模組（Arduino Mega 2560）

- 微控制器：Arduino Mega 2560（ATmega2560）
- 功能職責:
  - 每秒讀取 MQ2、MQ3、MQ9、MQ135、TGS2602、TGS2620 之氣體感測數值，以及 BME280 之溫度與氣壓資料
  - 格式化為 CSV 字串 (格式: "MQ2, MQ3, MQ9, MQ135, TGS2602, TGS2620, TempC, Humiditypct, PressurehPa")
  - 透過 USB Serial Port（115200 baud）傳送至 Raspberry Pi 3
  - 提供感測器電源與資料收集控制功能

單元 3: 邊緣運算模組(Raspberry Pi 3)

- 運算平台: Raspberry Pi 3 Model B
- 功能職責:
  - 接收 Arduino Serial 數據
  - 執行特徵工程計算
  - 載入 ExtraTrees 模型推論
  - 提供 Flask 模擬／測試介面
  - 支援 SSH 遠端管理

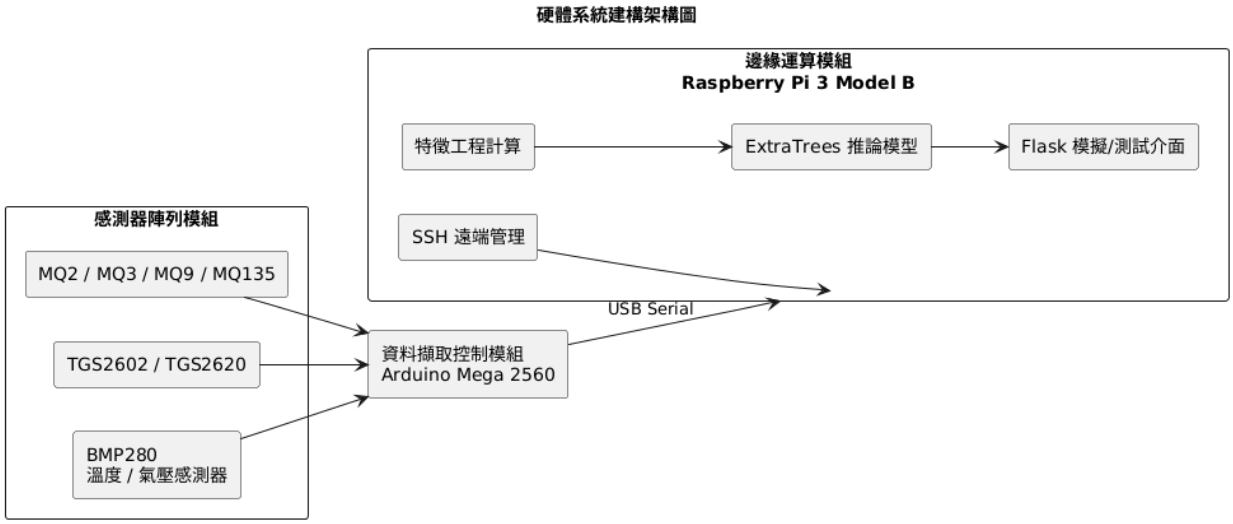


圖 3.1. 1.1 硬體系統建構架構圖

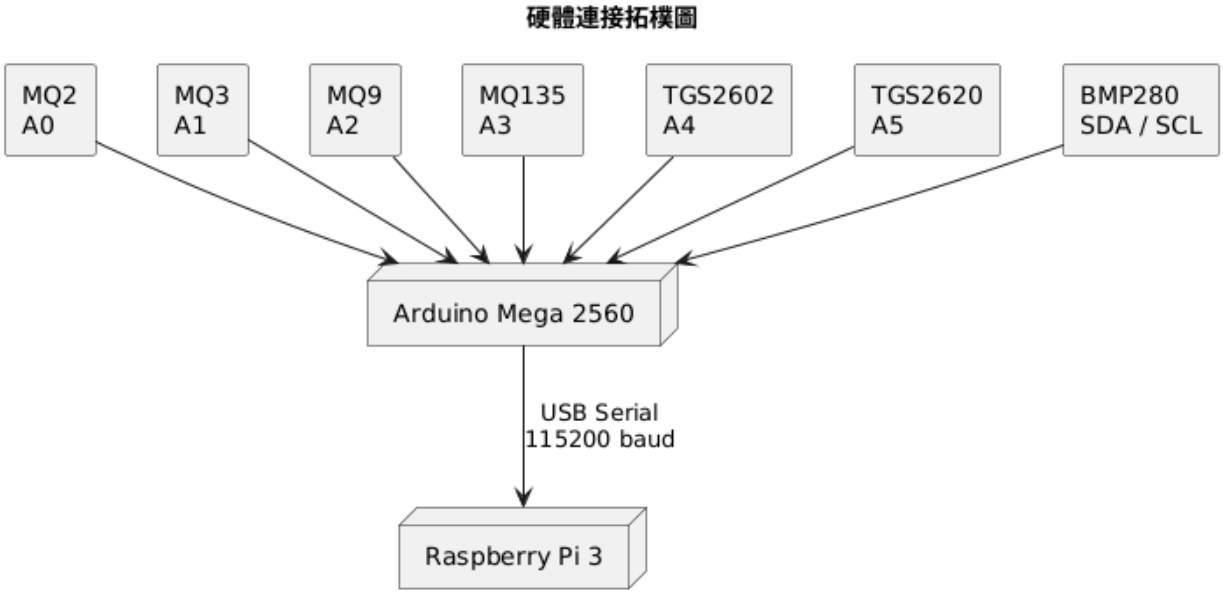


圖 3.1. 1.2 硬體連接拓樸圖

3.1.2 軟體建構架構

軟體系統採用模組化分層架構,分為五大層次:

Layer 1: 數據擷取層 (Data Acquisition Layer)

- Arduino 韌體模組:
  - sensor_read(): 讀取類比/數位感測器數值
  - format_csv(): 格式化為 CSV 字串
  - serial_write(): Serial 輸出至 Raspberry Pi 3

Layer 2: 數據前處理層 (Data Preprocessing Layer)

- Serial 通訊模組 (PySerial):
  - open_serial_port(): 開啟 Serial 連線
  - read_serial_data(): 讀取 CSV 數據流
  - parse_csv_line(): 解析 Arduino 傳送之感測資料
- 數據緩衝模組:
  - window_buffer[]: 30 秒滾動窗口緩衝區
  - append_data(): 新增數據至緩衝區
  - check_window_ready(): 檢查是否累積足夠數據

Layer 3: 特徵工程層 (Feature Engineering Layer)

- 基準線校正模組 (calibrate_air_30s.py):
  - collect_air_baseline(duration=30): 收集 30 秒空氣背景數據
  - compute_baseline_stats(): 計算各感測器均值/標準差

    - save_airbase_json(): 儲存至 airbase.json
- 特徵計算模組:
    - load_airbase(): 載入校正參數
    - compute_features(): 計算 11 項部署特徵
        - 歸一化特徵: MQ3stdnorm, MQ3rangenorm, TGS2602minnorm, MQ2aucnorm, MQ2meannorm, MQ9minnorm, TGS2602stdnorm
        - 比值特徵: MQ2MQ3ratio, MQ3MQ135ratio
        - 變化率特徵: MQ9slope, MQ9deltamean

Layer 4: 模型推論層 (Model Inference Layer)

- 模型載入模組:
    - load_model(deploystudent.pkl): 載入 ExtraTrees 模型
    - load_feature_columns(featurecolumns.json): 載入特徵名稱清單
    - load_deploy_meta(deploymeta.json): 載入 Stage 映射元資料
- 推論引擎模組 (inference_30s.py):
    - predict_proba(): 輸出 4 階段機率分布
    - apply_guard_baseline(): 應用基準線保護邏輯
    - apply_override_logic(): 針對 Stage 2→3 應用 TGS2602/MQ3 Override 規則
    - argmax_prediction(): 輸出最終成熟度預測

Layer 5: 應用服務層 (Application Service Layer)

- Flask 網頁服務模組 (applocal.py):
    - @app.route('/calibration'): 校正功能路由
    - @app.route('/inference'): 推論功能路由
    - ssh_execute_command(): 透過 Paramiko 執行 Raspberry Pi 遠端指令
    - display_probabilities(): 顯示 4 階段機率分布卡片
    - display_result(): 顯示最終成熟度預測

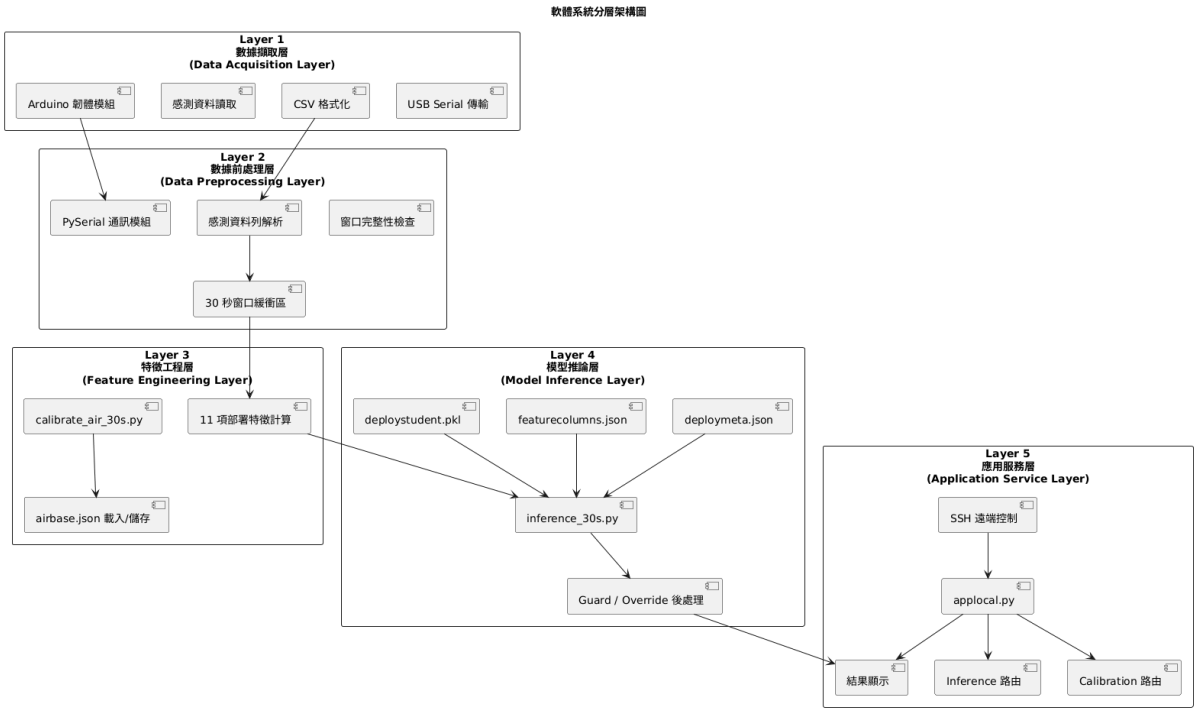


圖 3.1. 1.3 軟體分層架構圖

3.2 軟硬體組件設計說明

3.2.1 硬體組件詳細設計

1. 組件 A: 氣體感測器陣列

感測器型號	偵測物質	在鳳梨成熟度檢測中的角色
MQ-2	煙霧、液化氣、可燃性氣體	偵測鳳梨釋放之可燃性揮發物，作為成熟過程中揮發性氣體變化之輔助參考；其比值特徵可用於模型判讀。
MQ-3	酒精、乙醇	偵測成熟與過熟過程中之酒精類揮發物，為部署模型的重要特徵來源之一，例如 MQ3stdnorm、MQ3rangenorm。
MQ-9	一氧化碳、可燃氣體	偵測氧化反應相關氣體變化，可反映成熟過程中揮發物變動趨勢；其斜率與變化量特徵可作為成熟速率參考。
MQ-135	氨氣、苯、煙霧等揮發性氣體	偵測較廣泛之 VOC 與含氮揮發物，常作為與 MQ3 之間的

		比值特徵來源，例如 MQ3MQ135ratio。
TGS2602	VOC、氨氣、硫化氫	對低濃度 VOC 具高靈敏度，為本系統重要核心感測器之一；其最小值與標準差正規化特徵（如 TGS2602minnorm、TGS2602stdnorm）為部署模型關鍵特徵。
TGS2620	酒精、有機溶劑	對酒精與有機溶劑類揮發物具高靈敏度，可補強 MQ3 與其他氣體感測器之資訊，提升多感測器交叉驗證能力。

Table 7: 氣體感測器陣列規格與功能

設計考量:

- 採用 6 種氣體感測器交叉量測，以提升 VOC 特徵擷取之穩定性與辨識能力。
- TGS 系列靈敏度較高，MQ 系列量測範圍較廣，兩者混合配置可兼顧靈敏度與成本。
- 各感測器需經暖機後方可達到較穩定讀值，系統設計支援暖機參數調整。

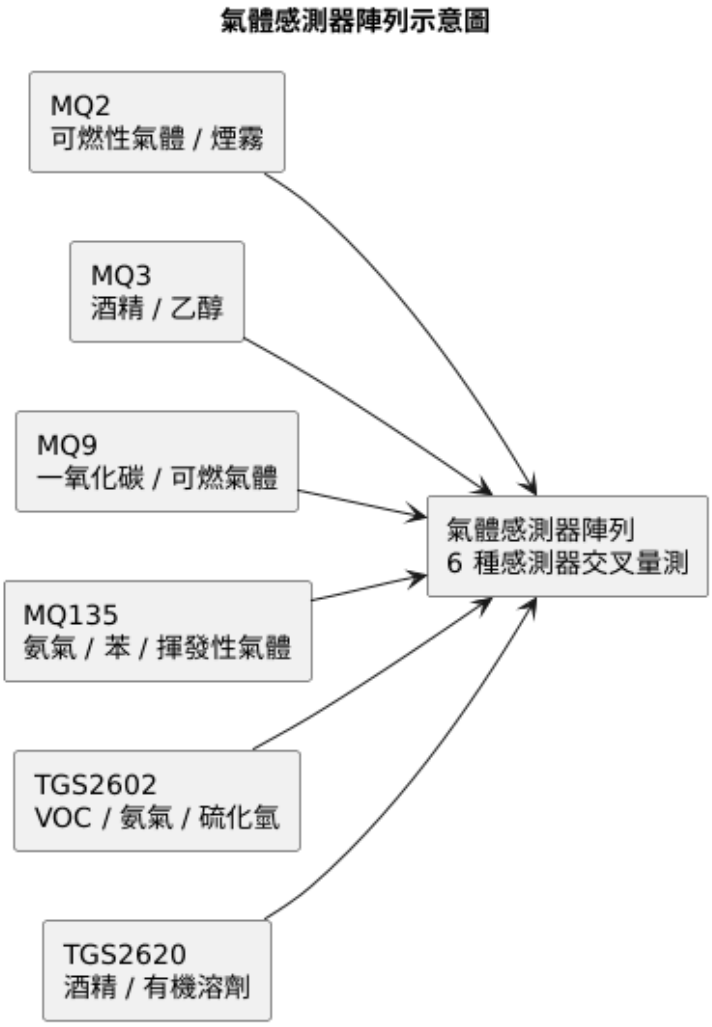


圖 3.2. 1.1 氣體感測器陣列示意圖

8. 組件 B: 環境感測器

感測器型號	量測參數	功能說明
BME280	溫度、濕度、氣壓	量測環境溫度與氣壓資訊，作為量測條件參考，並提供系統進行環境狀態監測。

Table 8: 環境感測器規格

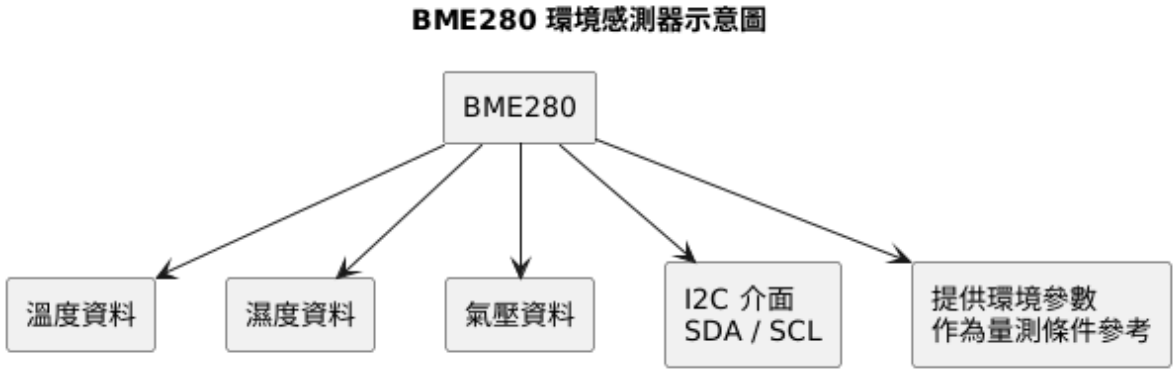


圖 3.2. 1.2 BME280 環境感測器示意圖

9. 組件 C: Arduino Mega 2560 控制單元

規格項目	參數
微控制器	ATmega2560（8-bit AVR 架構）
時脈頻率	16 MHz
Flash 記憶體	256 KB（韌體儲存）
SRAM	8 KB（資料暫存）
EEPROM	4 KB（參數儲存）
類比輸入	16 個 10-bit ADC 通道（本系統使用 A0～A5 連接 6 個氣體感測器）
數位 I/O	14 個腳位，SDA/SCL 連接 BME280
I2C	SDA / SCL，用於連接 BME280
Serial 通訊	USB Serial 與 Raspberry Pi 3 連接

Table 9: Arduino Mega 2560 規格

韌體設計重點:

- 每秒執行一次感測資料讀取（1 Hz 採樣率）。
- 類比訊號讀值採 10-bit ADC（0～1023 範圍）。
- 將感測資料格式化為 CSV 字串後透過 USB Serial 傳送至 Raspberry Pi 3。
- 支援 30 秒視窗資料累積，供後續特徵工程與模型推論使用。

圖 3.2. 1.3 Arduino Mega 2560 控制單元功能圖

10. 組件 D:

規格項目	參數
處理器	Broadcom BCM2837，1.2 GHz 64-bit Quad-Core ARM Cortex-A53
記憶體	1 GB LPDDR2 SDRAM
網路介面	10/100 Ethernet、802.11n WiFi
USB 埠	4 個 USB 2.0（其中一個連接 Arduino）
GPIO	40-pin 擴充接腳(本專案未使用)
儲存空間	MicroSD 卡（本系統使用 32 GB microSD 卡）
作業系統	Raspberry Pi OS（Debian-based Linux）

Raspberry Pi 3 運算單元

效能考量:

- 由於 Raspberry Pi 3 僅具 1 GB RAM，因此部署端採用較輕量之 ExtraTrees 模型，而非較大型之訓練模型。
- Raspberry Pi 3 負責接收 Arduino 傳來之感測資料、執行 30 秒視窗特徵計算、載入部署模型並完成成熟度推論。
- 系統亦支援 SSH 遠端管理；Flask 介面僅作為開發、測試與展示用途，正式展示以 App 為主。

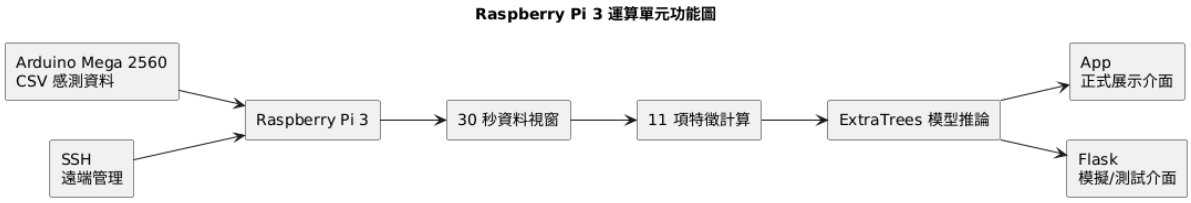


圖 3.2. 1.4 Raspberry Pi 3 運算單元功能圖

以下是本節之總體硬體組件關係圖、感測器接線拓樸圖:

圖 3.2. 1.5 硬體組件關係圖

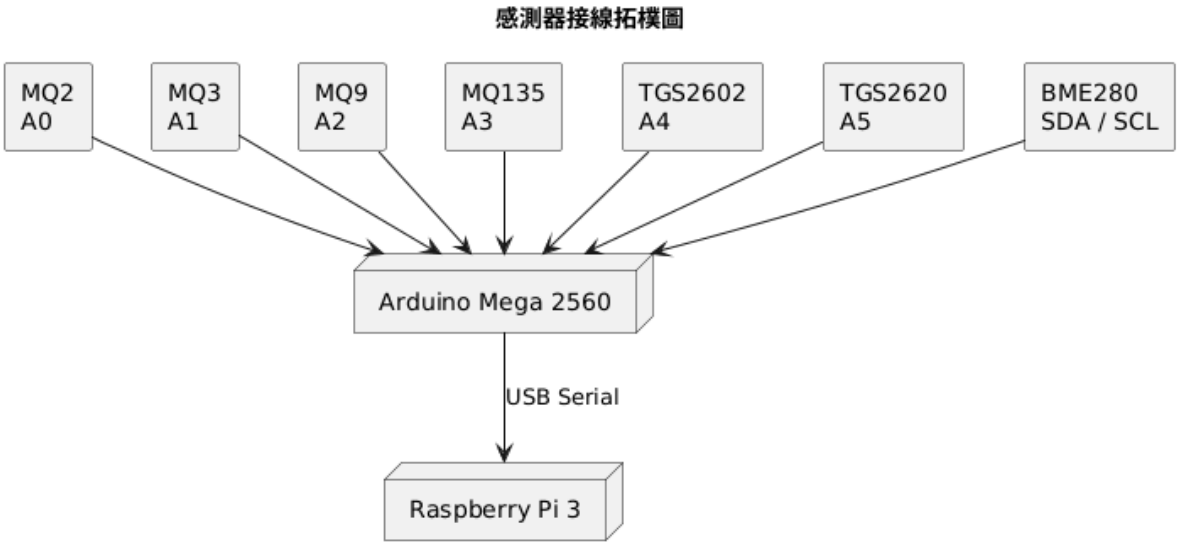


圖 3.2. 1.6 感測器接線拓樸圖

3.2.2 軟體組件詳細設計

1. 組件 E: ExtraTrees 機器學習模型

模型參數	設定值
演算法	ExtraTrees (Extremely Randomized Trees)
輸入特徵	11 項部署特徵向量
輸出類別	4 類成熟度 (Stage 0、1、2、3)
模型大小	6.2 MB (deploystudent.pkl)
訓練策略	Teacher-Student 知識蒸餾 (Teacher: CatBoost；Student: ExtraTrees)
訓練數據	14 顆鳳梨，333 筆 per-day features，經 Pseudo Labeling 擴充至 69 筆 Stage 3 數據
效能指標	Mean Accuracy: 86.15%, ，Macro F1: 83.09%, ，Sample Accuracy: 79.27%

Table 11: ExtraTrees 模型規格

模型訓練流程:

(1) 收集鳳梨 VOC 量測資料，建立 4 階段成熟度標籤。

(2) 進行特徵工程，建立 PID-specific 與 shared-by-date 特徵。

(3) 針對 Stage 3 樣本不足問題，透過 Pseudo Labeling 擴充樣本數量。

(4) 使用 CatBoost 作為 Teacher 模型進行訓練與交叉驗證。

(5) 將知識蒸餾至 ExtraTrees Student 模型，作為部署端使用之輕量模型。

(6) 透過 Mutual Information 挑選部署所需之核心特徵。

(7) 配合 Per-PID 校正與後處理規則，提升部署端樣本準確率。

部署特徵清單 (featurecolumns.json):

No.	特徵名稱	計算方式
1	MQ3stdnorm	MQ3 標準差歸一化
2	MQ3rangenorm	MQ3 範圍歸一化
3	MQ2MQ3ratio	MQ2/MQ3 濃度比值
4	MQ3MQ135ratio	MQ3/MQ135 濃度比值
5	MQ9slope	MQ9 濃度變化斜率
6	TGS2602minnorm	TGS2602 最小值歸一化
7	MQ2aucnorm	MQ2 曲線下面積歸一化
8	MQ2meannorm	MQ2 均值歸一化
9	MQ9minnorm	MQ9 最小值歸一化
10	TGS2602stdnorm	TGS2602 標準差歸一化
11	MQ9deltamean	MQ9 均值變化量

Table 12: 部署特徵清單

圖 3.2. 1.7 ExtraTrees 機器學習模型示意圖

2. 組件 F: 基準線校正模組 (calibrate_air_30s.py)

功能說明:

校正模組負責於無鳳梨情況下收集 30 秒空氣背景資料，計算各感測器之基準統計量，並輸出為基準線設定檔，供後續部署推論時進行特徵歸一化與 Guard Baseline 判斷使用。

執行流程:

(1) 開啟 Serial Port 連線

(2) 執行 60-180 秒感測器暖機(可透過--warmup-sec 參數調整)

(3) 收集 30 秒空氣背景數據(無鳳梨存在)

(4) 計算各感測器的均值(mean)、標準差(std)、最小值(min)、最大值(max)

(5) 儲存至 airbase.json

輸出格式 (airbase.json):

```
{
"MQ2": {"mean": 450.2, "std": 12.3, "min": 420, "max": 480},
"MQ3": {"mean": 380.5, "std": 15.1, "min": 350, "max": 410},
...
}
```

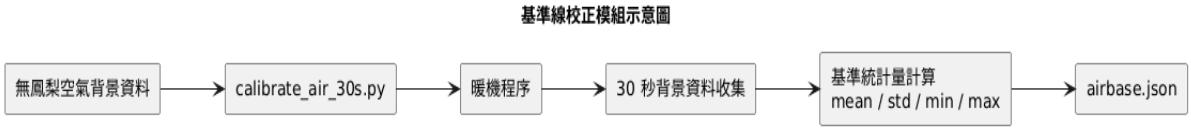


圖 3.2. 1.8 基準線校正模組示意圖

3. 組件 G: 推論引擎模組 (inference_30s.py)

功能說明:

推論引擎模組負責即時讀取 Arduino 傳來之感測資料，累積 30 秒視窗後計算部署特徵，載入 ExtraTrees 模型進行成熟度推論，並套用 Guard 與 Override 後處理規則，輸出最終成熟度結果。

執行流程:

(1) 初始化階段:

- 載入 deploystudent.pkl 模型
- 載入 featurecolumns.json 特徵名稱
- 載入 deploymeta.json Stage 映射
- 載入 airbase.json 校正參數

(2) 暖機階段:

- 執行感測器暖機(預設 30 秒,可調整)

(3) 數據收集階段:

- 開啟 Serial Port 連線
- 累積 30 秒數據窗口(約 30 筆資料)

(4) 特徵計算階段:

- 計算 11 項特徵向量
- 使用 airbase.json 進行歸一化

(5) 模型推論階段:

- ExtraTrees.predict_proba()輸出 4 階段機率分布

(6) 後處理階段:

- 應用 Guard Baseline 邏輯(保護基準線)
- 應用 Override 邏輯(針對 Stage 2→3,檢查 TGS2602 與 MQ3 特徵)
- argmax 取得最終預測 Stage

(7) 輸出階段:

- 顯示 4 階段機率分布
- 顯示最終成熟度預測(Stage 0/1/2/3)

Guard Baseline 邏輯:

若所有感測器數值接近 airbase.json 基準線(允許誤差範圍內),則強制輸出 Stage 0(未成熟)。

Override 邏輯:

若模型預測為 Stage 2,但 TGS2602 與 MQ3 特徵超過特定閾值,則 Override 為 Stage 3(過熟)。

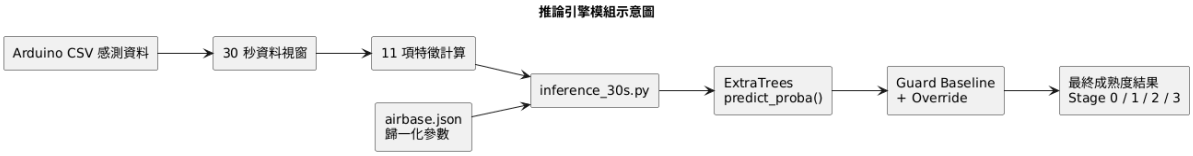


圖 3.2. 1.9 推論引擎模組示意圖

4. 組件 H: Flask 網頁應用 (applocal.py)

功能說明:

本模組提供系統於開發、測試與展示階段之模擬操作介面，可透過網頁觸發校正與推論功能，並顯示對應結果。正式展示介面以 App 為主，Flask 僅作為模擬／測試用途。

功能模組:

路由	功能說明
/（首頁）	顯示系統簡介與功能選單
/calibration	提供校正功能介面，透過 SSH 觸發 calibrate_air_30s.py，並顯示基準線結果
/inference	提供推論功能介面，透過 SSH 觸發 inference_30s.py，並顯示機率分布與最終預測結果

Table 13: Flask 模擬介面功能

SSH 遠端控制設計 (Paramiko):

- 連線至 Raspberry Pi (SSH Port 22)
- 執行 Python 腳本: python3 calibrate_air_30s.py 或 python3 inference_30s.py
- 擷取標準輸出(stdout)回傳至網頁顯示

介面設計:

- Calibration 介面: 顯示暖機進度、30 秒校正進度條、airbase.json 結果預覽
- Inference 介面: 顯示 4 階段機率卡片(視覺化長條圖)、最終預測結果(Stage 0-3 對應文字說明)

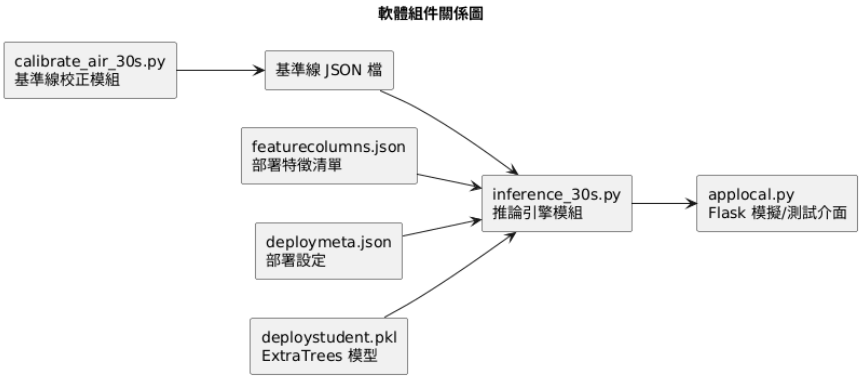


圖 3.2. 2.0 軟體組件關係圖

3.3 介面設計說明

本節說明本系統之介面架構、各頁面模組、UI 設計原則與雙語設計方式，作為系統開發、測試與成果展示之依據。本系統介面分為消費端 App 與農民端 App 兩端，分別對應一般消費者與農民／集貨場之使用情境；此外，於系統開發與驗證階段，另建置 Flask 網頁模擬介面作為功能測試與流程驗證用途。

3.3.1 介面結構概述

系統介面架構分為消費端 App 與農民端 App 兩端，各自對應不同使用者需求與操作流程。消費端 App 主要針對一般消費者，提供單顆鳳梨掃描、熟度辨識與互動式資訊呈現；農民端 App 主要針對農民與集貨場，提供批次掃描、批次管理與統計分析等功能。

整體流程由首頁啟動，依使用者身分與需求進入不同操作情境。消費端流程自首頁開始，經掃描、結果顯示至歷史紀錄與知識內容瀏覽；農民端流程則由首頁進入批次建立、批次掃描、結果彙整、報表分析與批次歷史管理。相關操作流程分別以消費端活動圖（Activity Diagram）與農民端活動圖（Activity Diagram）表示，用以描述兩端從首頁啟動至結果顯示與歷史／報表管理之完整操作脈絡。

3.3.2 各頁面模組設計

以下依模組方式說明各頁面設計。每一模組包含功能名稱、輸入、輸出、功能描述及 UI 顯示方式，並分為消費端模組與農民端模組兩部分。

A. 消費端模組

1. 首頁 (Home Page)

- 功能名稱：啟動掃描與語言切換
- 輸入：使用者點擊、語言選擇
- 輸出：導向掃描操作、語言狀態更新
- 功能描述：作為消費端主要入口，提供開始掃描按鈕與中英語言切換功能，使使用者可快速進入檢測流程。

- UI 顯示：中央大按鈕「Start Scan / 開始掃描」，下方語言切換開關，上方選項與鳳梨圖示。

2. 掃描流程頁 (Scan Process Page)
   - 功能名稱：引導放入與感測流程
   - 輸入：使用者確認放入、感測器資料
   - 輸出：感測資料傳送 API、轉入處理流程
   - 功能描述：引導使用者將鳳梨放置於感測裝置中，透過動畫倒數與進度顯示輔助完成量測，並於感測完成後將資料傳送至 API。
   - UI 顯示：語音提示圖示、放入鳳梨插圖、30 秒動畫倒數、進度條。
3. 結果頁 (Result Page)
   - 功能名稱：顯示檢測結果
   - 輸入：API 回傳 JSON 資料
   - 輸出：熟度分類、信心值、食用建議、儲存至歷史紀錄
   - 功能描述：解析後端 API 回傳結果，將鳳梨熟度分類以顏色、文字與圖示方式顯示，並提供重新掃描與歷史查看功能。
   - UI 顯示：大字熟度標籤（綠/黃/橙/紅）、信心值進度條、食用建議文字、歷史按鈕。
4. 歷史紀錄模組 (History Module)
   - 功能名稱：檢視個人歷史
   - 輸入：使用者選擇日期
   - 輸出：歷史檢測清單顯示
   - 功能描述：提供使用者查閱過往單筆檢測結果，方便追蹤曾掃描之鳳梨熟度資訊。
   - UI 顯示：清單卡片（日期、熟度、信心值）。
5. 知識內容模組 (Knowledge Module)
   - 功能名稱：鳳梨知識展示
   - 輸入：使用者點擊選擇
   - 輸出：知識頁面顯示
   - 功能描述：提供與鳳梨選購、保存與食用相關之教育內容，提升系統互動性與知識傳遞價值。
   - UI 顯示：圖文卡片、互動式問答內容。

另以消費端循序圖（Sequence Diagram）呈現消費端各頁面與 API、感測模組之互動次序，以說明資料傳遞與畫面切換邏輯。

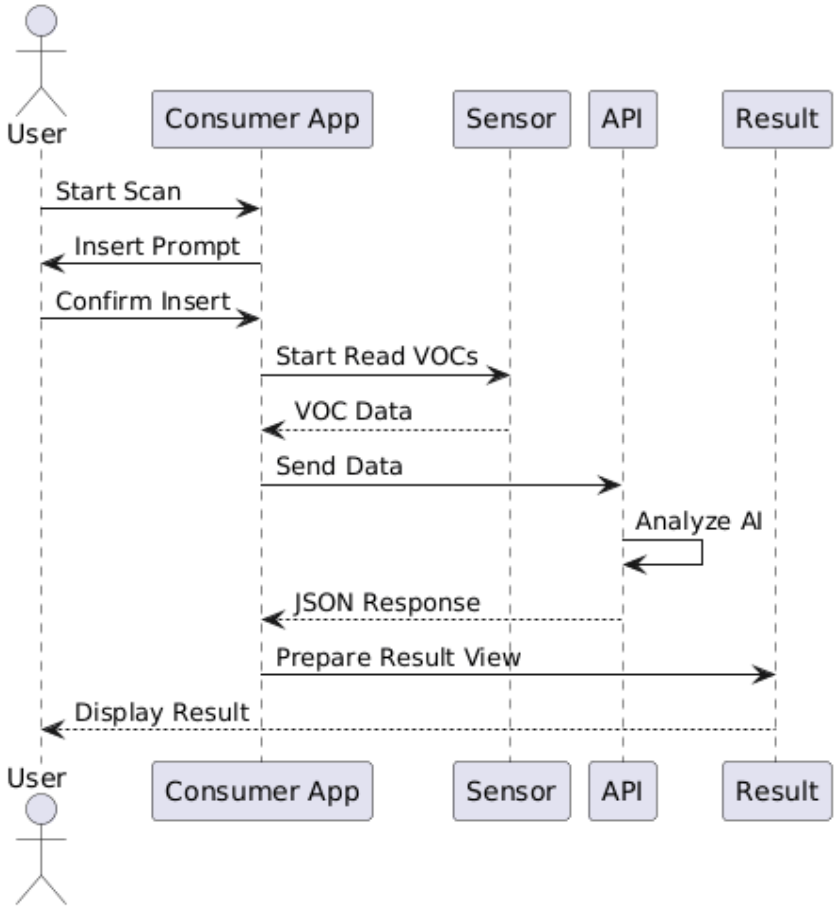


圖 3.3. 1.1 消費端循序圖（Sequence Diagram）

B. 農民端模組

1. 農民首頁 (Farmer Home Page)
   - 功能名稱：批次管理入口
   - 輸入：使用者選擇新建批次或歷史紀錄
   - 輸出：導向批次建立或歷史頁面
   - 功能描述：作為農民端主要入口，顯示目前批次概況與統計摘要，並提供快速操作功能。
   - UI 顯示：批次清單、統計摘要、新建按鈕。
2. 批次建立模組 (Batch Create Module)
   - 功能名稱：建立批次資訊
   - 輸入：批次名稱、數量、日期
   - 輸出：批次 ID 生成
   - 功能描述：記錄單一批次之基本資料，作為後續批次掃描與統計分析之依據。
   - UI 顯示：表單輸入欄位、確認按鈕。
3. 批次掃描模組 (Batch Scan Module)

- 功能名稱：批次感測流程
- 輸入：批次 ID、連續感測資料
- 輸出：批次資料上傳 API
- 功能描述：支援多顆鳳梨之連續掃描流程，於每次掃描後記錄結果並納入同一批次。
- UI 顯示：進度條、剩餘數量計數器。

4. 批次結果模組 (Batch Result Module)
    - 功能名稱：批次結果彙整
    - 輸入：API 批次回傳
    - 輸出：批次統計顯示
    - 功能描述：統整整批鳳梨之熟度分佈、平均值與整體結果，協助農民快速掌握批次品質。
    - UI 顯示：餅圖、表格摘要。
5. 報表分析模組 (Report Analysis Module)
    - 功能名稱：生成報表
    - 輸入：批次選擇
    - 輸出：PDF/圖表匯出
    - 功能描述：針對指定批次產出熟度統計與趨勢分析報表，支援後續管理與決策參考。
    - UI 顯示：圖表面板、匯出按鈕。
6. 批次歷史模組 (Batch History Module)
    - 功能名稱：批次歷史管理
    - 輸入：日期篩選條件
    - 輸出：歷史批次清單
    - 功能描述：供使用者檢視、搜尋與管理過往批次紀錄，必要時可刪除不再需要之歷史資料。
    - UI 顯示：時間軸清單、搜尋功能。

另以農民端循序圖（Sequence Diagram）呈現農民端從批次建立、掃描、結果彙整至報表匯出之系統互動流程。

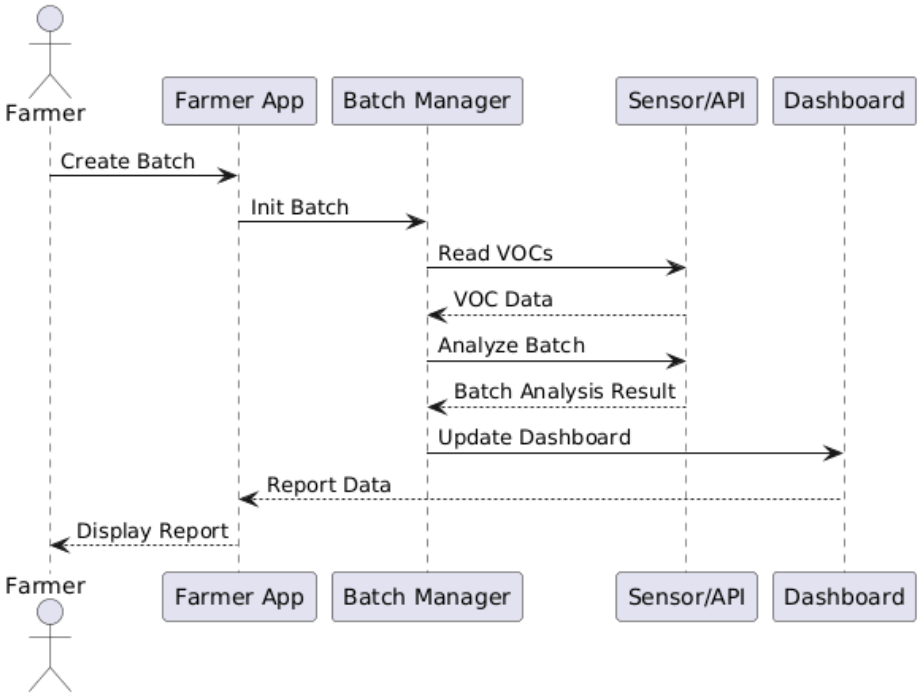


圖 3.3. 1.2 農民端循序圖（Sequence Diagram）

3.3.3 UI 設計原則

1. 本系統介面設計依不同使用者情境訂定下列原則：
2. 農民友善：農民端介面採大按鈕、高對比配色與大字體設計，以因應戶外強光、快速操作及配戴手套時之使用需求。
3. 圖像化呈現：以鳳梨圖示與顏色碼作為熟度辨識輔助，其中綠色代表未熟、黃色代表初熟、橙色代表完熟、紅色代表過熟，以降低文字理解負擔。
4. 低學習成本：介面操作流程力求單純明確，避免複雜子選單，使使用者於首次使用時即可快速完成任務。
5. 依端別調整重點：消費端介面偏向直覺化、教育導向與互動式體驗；農民端則偏向效率、批次管理、資訊摘要與統計可讀性，以符合不同使用需求。

3.3.4 雙語設計

本系統之消費端與農民端皆支援中英雙語介面。語系切換採用設定區上方之滑動開關（中文／English），切換後可即時更新所有 UI 文字與語音提示，並將使用者偏好儲存於裝置本地，以保留下次啟動時之語系設定。

雖然消費端與農民端皆採一致的語系切換策略，但在文字內容設計上，仍依不同使用者需求進行調整。消費端文字偏向簡潔、易懂與具引導性；農民端文字則偏向資訊完整、操作明確與統計解讀導向，以提升不同情境下之使用效率。

3.3.5 Flask 模擬介面與驗證流程

除正式之消費端 App 與農民端 App 外，本系統於開發與測試階段另建置 Flask 單頁式模擬介面，用以驗證感測器量測、空氣基線校正、模型推論與結果顯示流程。正式操作介面以 App 為主，Flask 僅作為功能模擬、測試與展示用途。

(1) 介面結構概述

系統採單頁式（Single Page）介面設計，所有主要操作集中於同一頁面，區分為下列區塊：

- 系統標題與狀態列：顯示系統名稱與目前操作狀態，例如待機中、校正中、推論中、推論完成、尚未校正或錯誤狀態等。
- 校正控制區：提供「Air Baseline 校正」按鈕與暖機秒數選擇（0／60／180 秒）。
- 推論控制區：提供「開始推論」按鈕與暖機秒數選擇，啟動 30 秒窗口量測與推論程序。
- 結果顯示區：以卡片方式呈現 Stage 0～Stage 3 機率與最終成熟度判定結果。
- 訊息與錯誤提示區：顯示校正／推論過程之成功、失敗與錯誤訊息。

(2) 主要操作元件

- Air Baseline 校正按鈕：觸發 calibrate_air_30s.py，完成空氣基線量測與 airbase.json 產生。
- 開始推論按鈕：觸發 inference_30s.py，執行 30 秒窗口 VOC 量測、特徵計算與模型推論。
- 成熟度結果卡片：顯示四階段機率（Stage 0～Stage 3）與最終預測 Stage，並附文字說明建議。
- 狀態訊息列：顯示目前作業階段與錯誤訊息，如「尚未校正」、「Serial 連線失敗」等。

(3) 介面互動流程

- 使用者透過瀏覽器連線至 Flask 網頁（同一區域網路）。
- 先於校正控制區執行 Air Baseline 校正，待狀態列顯示「校正完成」後，方可進行推論操作。
- 將待測鳳梨放置於感測器陣列前方，於推論控制區設定暖機秒數並按下「開始推論」。
- 系統完成 30 秒量測與推論後，於結果顯示區更新各 Stage 機率與最終成熟度預測，並於狀態列標示「推論完成」。

(4) 驗證功能與測試用途

Flask 模擬介面除提供基本操作外，亦作為系統整合測試與功能驗證平台，其主要用途如下：

- 驗證空氣基線流程：確認 calibrate_air_30s.py 是否成功執行，並正確產生 airbase.json。

- 驗證感測資料讀取：確認感測器資料是否可正常接收並進入後續處理流程。
- 驗證推論流程：確認 inference_30s.py 是否能完成 30 秒量測、特徵計算與模型推論。
- 驗證結果顯示：確認模型輸出之 Stage 0～Stage 3 機率與最終成熟度結果是否能正確顯示於前端。
- 驗證錯誤處理機制：當系統發生尚未校正、Serial 連線失敗、資料不足或程式執行異常等情形時，確認介面是否能即時顯示錯誤訊息，協助開發者定位問題並進行修正。

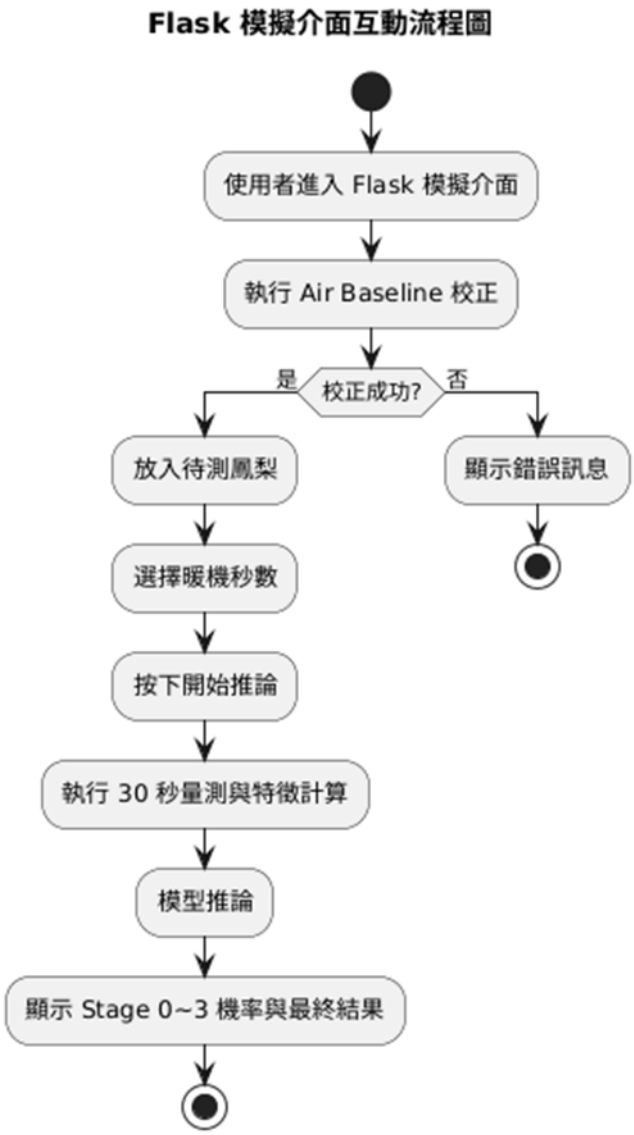


圖 3.3. 1.1 介面互動流程圖

3.4 作業程序設計說明

本節定義系統部署、校正、成熟度檢測與維護等作業程序，作為操作手冊與測試程序之依據。

3.4.1 系統部署作業流程

步驟 1：硬體組裝與連接

- 將 6 顆氣體感測器分別接至 Arduino Mega 2560 類比腳位（A0～A5）。
- 將 BME280 透過 I2C（SDA／SCL）接至 Arduino Mega 2560。
- 透過 USB 線連接 Arduino Mega 2560 與 Raspberry Pi 3。
- 為 Raspberry Pi 3 接上電源（5V 2.5A）與網路（乙太網路或 Wi-Fi）。

步驟 2：Arduino 韌體燒錄

- 於開發電腦安裝 Arduino IDE，撰寫感測器讀取與 Serial 輸出韌體。
- 透過 USB 將韌體上傳至 Arduino Mega 2560。
- 以 Serial Monitor 驗證 CSV 格式之感測資料輸出是否正確。

步驟 3：Raspberry Pi 3 環境設置

- 於 MicroSD 卡安裝 Raspberry Pi OS，插入 Raspberry Pi 3 後開機。
- 設定 Wi-Fi／Ethernet 網路連線並啟用 SSH 服務（sudo raspi-config）。
- 安裝 Python 3 與必要套件：
- sudo apt-get update
- sudo apt-get install python3-pip
- pip3 install scikit-learn numpy pandas flask paramiko pyserial

步驟 4：部署檔案傳輸

- 透過 SCP 或 FTP 工具將下列檔案傳至 Raspberry Pi 3 指定目錄：
- deploystudent.pkl（約 6.2 MB）
- featurecolumns.json、deploymeta.json、airbase.json（若已有）
- calibrate_air_30s.py、inference_30s.py、applocal.py
- 設定執行權限：chmod +x *.py。

步驟 5：Serial Port 設定

- 於 Raspberry Pi 3 上查詢 Arduino Serial Port 名稱：
- ls /dev/ttyUSB* 或 ls /dev/ttyACM*
- 修改 Python 腳本中 Serial Port 路徑（例如 /dev/ttyUSB0）。
- 將使用者加入 dialout 群組以取得序列埠權限：
- sudo usermod -a -G dialout $USER

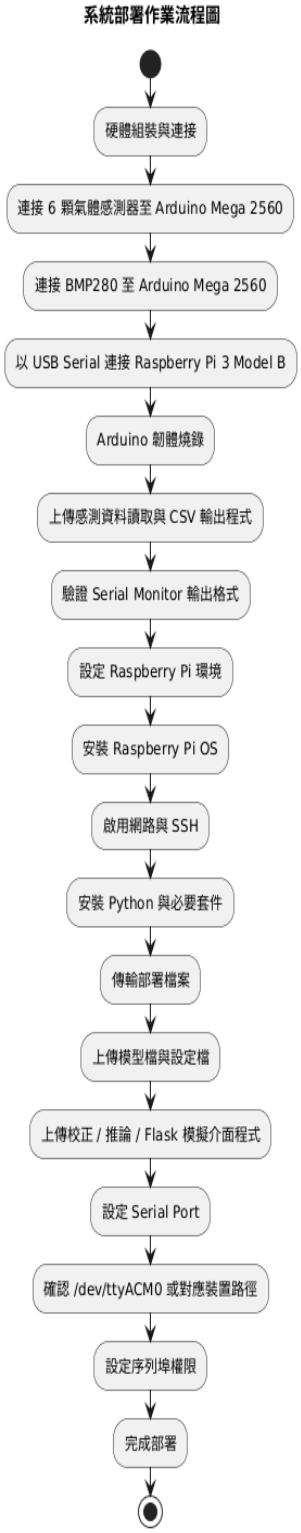


圖 3.4. 1.1 系統部屬流程圖

3.4.2 系統校正作業流程(空氣基線校正)

執行時機:

- 系統首次啟動。
- 更換氣體感測器或 BME280 後。
- 環境溫濕度大幅變化時。
- 每週定期校正(建議)。

操作步驟:

1. 準備階段:
   - 確保測試環境中無鳳梨或其他揮發性物質。
   - 環境溫度 20-30°C,相對濕度 40-70%。
2. 執行校正:
   - 方法 A (命令列): SSH 連線至 Raspberry Pi 3，執行 python3 calibrate_air_30s.py --warmup-sec 60。
   - 方法 B (網頁介面): 開啟 Flask 應用，點選「Air Baseline 校正」按鈕。
3. 暖機等待:
   - 系統依設定之暖機秒數(預設 60 秒,可調整至 180 秒)完成暖機。
   - 此階段感測器加熱至穩定工作溫度。
4. 數據收集:
   - 暖機後自動收集 30 秒空氣背景數據，並顯示「收集 30 秒空氣 baseline」。
   - 收集 30 秒空氣背景數據。
5. 結果驗證:
   - 檢查 airbase.json 是否成功生成，，並確認各感測器均值落於合理範圍。
   - 檢查各感測器均值、標準差與最小／最大值是否落於合理範圍，且與過往校正結果相比無明顯異常漂移。
   - 若數值異常,檢查感測器連接或重新執行校正。
   - 校正完成後，系統將輸出 airbase.json 作為後續特徵歸一化與 Guard Baseline 判斷之依據。

airbase.json 範例:

{
"MQ2": {"mean": 450.2, "std": 12.3, "min": 420, "max": 480},
"MQ3": {"mean": 380.5, "std": 15.1, "min": 350, "max": 410},
"MQ9": {"mean": 520.8, "std": 18.7, "min": 490, "max": 560},
"MQ135": {"mean": 410.3, "std": 14.2, "min": 380, "max": 440},
"TGS2602": {"mean": 550.1, "std": 22.5, "min": 510, "max": 590},
"TGS2620": {"mean": 470.6, "std": 16.8, "min": 440, "max": 510}
}

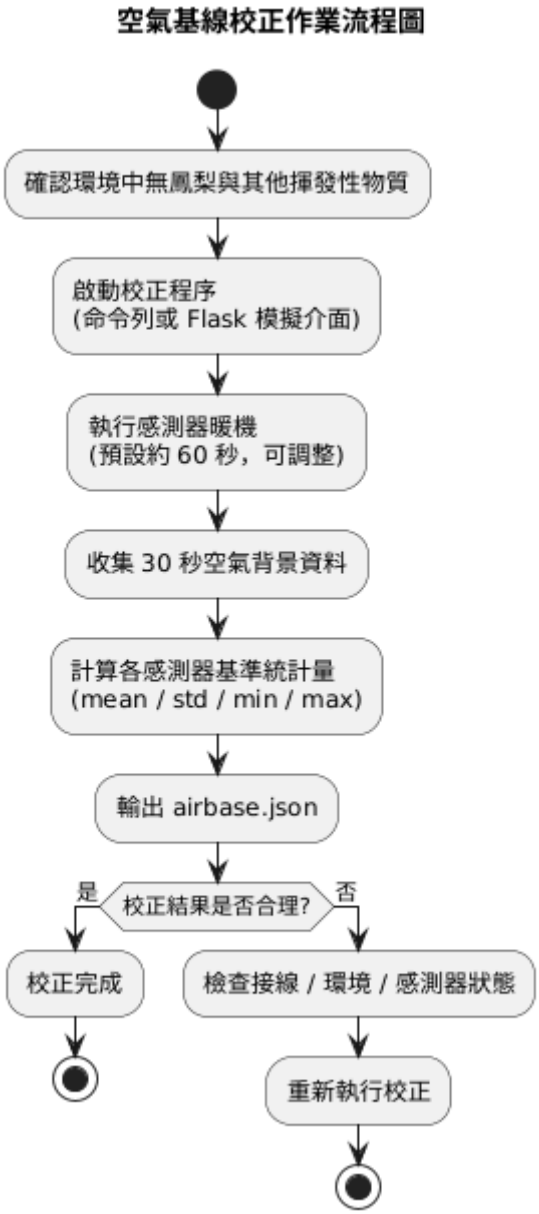


圖 3.4. 1.2 空氣基線校正作業流程圖

3.4.3 成熟度檢測作業流程

操作步驟:

1. 鳳梨準備:
   - 選取待檢測鳳梨，確保表面清潔、無明顯外傷。
   - 將鳳梨置於感測器陣列前方適當位置，建議距離約 5 公分，以利量測其揮發性氣體。
2. 執行推論:
   - 方法 A (命令列): SSH 連線，執行 python3 inference_30s.py --warmup-sec 30。
   - 方法 B (網頁介面): 開啟 Flask 應用，點選「開始推論」按鈕。
3. 暖機等待:
   - 系統執行感測器暖機(預設 30 秒)。
4. 數據收集:
   - 系統於資料收集階段顯示量測中之狀態訊息，例如前置累積 0 秒，兩側窗口 30 秒」。
   - 累積 30 秒鳳梨 VOC 數據。
5. 特徵計算:
   - 系統自動計算 11 項特徵。
   - 載入 airbase.json 進行歸一化。
6. 模型推論:
   - ExtraTrees 模型輸出四階段機率分布，並套用 Guard Baseline 與 Override 規則。
   - 顯示機率卡片:
     - Stage 0: XX.XX%
     - Stage 1: XX.XX%
     - Stage 2: XX.XX%
     - Stage 3: XX.XX%
7. 後處理:
   - 應用 Guard Baseline 與 Override 邏輯
   - 輸出最終預測結果:
     - Stage 0: 未成熟 (建議再等待 3-5 天)
     - Stage 1: 微熟 (建議再等待 1-2 天)
     - Stage 2: 適食成熟 (建議立即食用)
     - Stage 3: 過熟 (建議盡快食用,避免腐敗)
8. 結果記錄:

- 推論完成後，結果可同步傳送至 App 介面顯示；開發與測試階段則可由 Flask 模擬介面顯示。
- 人工記錄檢測結果與實際成熟度(用於後續模型改進)
- 供後續模型驗證、誤差分析與版本優化使用。

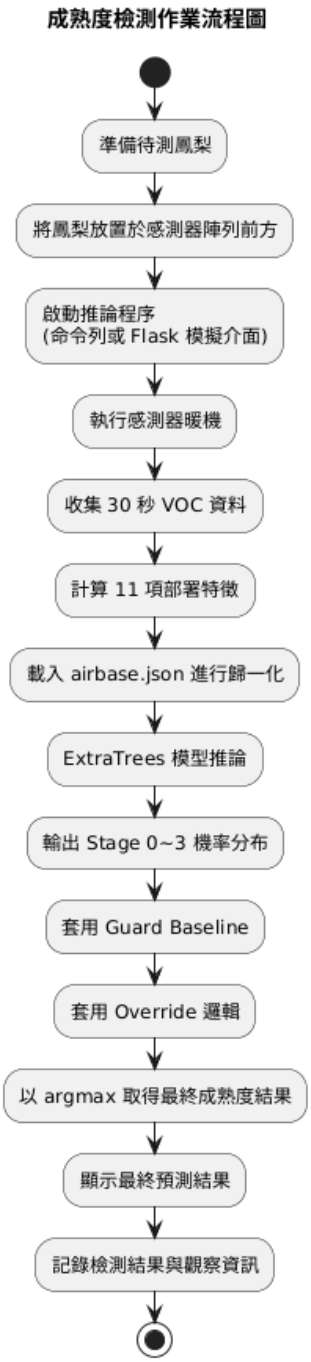


圖 3.4. 1.3 成熟度檢測作業流程圖

3.4.4 系統維護作業流程

日常維護:

- 每日檢查: 確認 Arduino Mega 2560 與 Raspberry Pi 3 電源供應正常，網路連線穩定，且 USB Serial 連線未鬆脫。
- 空氣基線校正：建議每週至少執行一次，或於更換感測器、環境條件明顯改變時重新執行。
- 感測器清潔: 每月以乾布輕拭感測器表面，避免灰塵累積。
- 定期維護:
  - 每季度:
    - 檢查感測器靈敏度(使用標準氣體樣本驗證)
    - 備份系統檔案(deploystudent.pkl、airbase.json、歷史數據)
    - 更新 Raspberry Pi 作業系統與 Python 套件
  - 每半年:
    - 檢查 Arduino 與 Raspberry Pi 硬體狀況
    - 評估模型準確率，考慮是否需重新訓練
- 故障排除:
  - Serial 通訊失敗:
    - 檢查 USB 連接
    - 確認 Serial Port 裝置路徑（如 /dev/ttyACM0）是否正確。
    - 確認使用者是否已加入 dialout 群組，以取得序列埠存取權限。
    - 重新插拔 USB Serial 連線，必要時重新啟動 Arduino Mega 2560 與 Raspberry Pi 3。
  - 模型推論錯誤:
    - 確認 airbase.json 存在且有效
    - 檢查 deploystudent.pkl 是否損壞
    - 驗證 Python 套件版本相容性
  - 感測器讀數異常:
    - 檢查感測器接線
    - 延長暖機時間至 180 秒
    - 考慮更換老化感測器

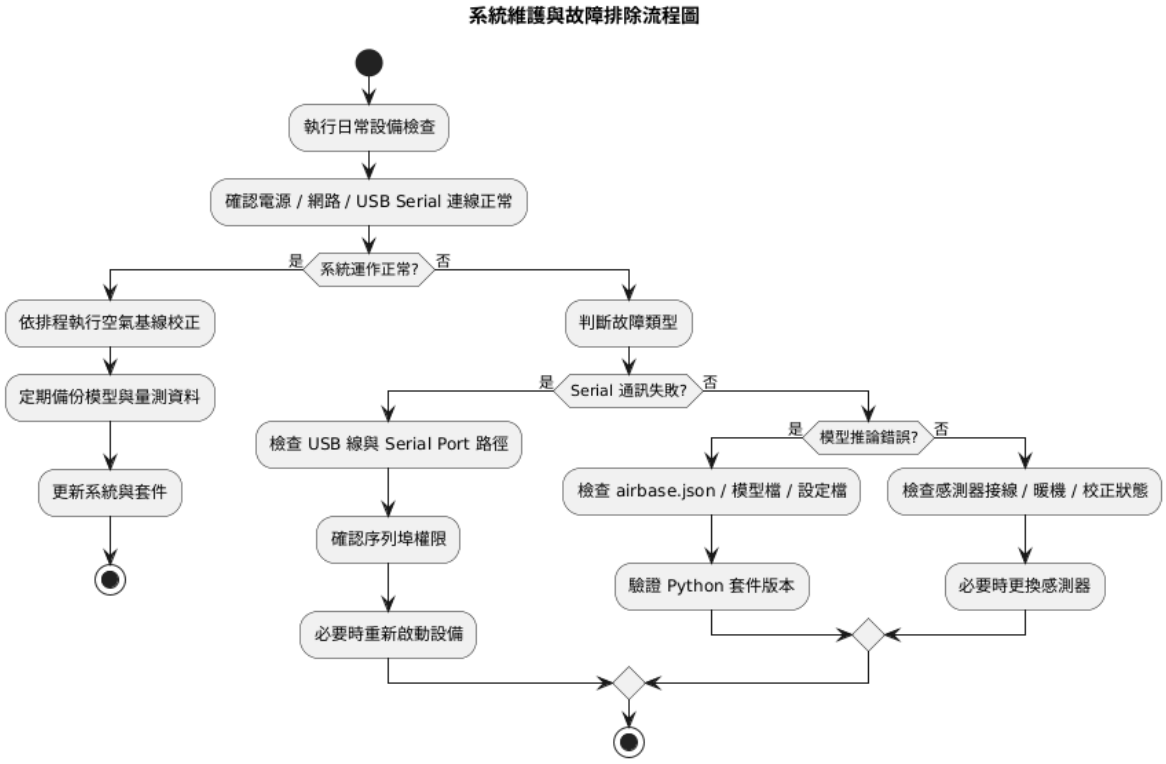


圖 3.4. 1.4 維護與故障排除流程圖

3.4.5 End-to-End 完整作業流程

整合測試流程:

1. Arduino Mega 2560 讀取感測器 VOC 資料，並以 CSV 格式輸出至 Raspberry Pi 3。
2. Raspberry Pi 3 累積 30 秒資料視窗，並計算 11 項部署特徵。
3. 系統載入 airbase.json 進行特徵歸一化，並使用 ExtraTrees 模型執行成熟度推論。
4. 輸出四階段機率分布，並套用 Guard Baseline 與 Override 後處理邏輯。
5. 系統將最終成熟度結果輸出至展示介面；Flask 介面作為模擬／測試用途，正式使用情境則由 App 顯示結果其中消費端 App 以單筆結果顯示與歷史查詢為主，農民端 App 則支援批次彙整與統計分析。

效能指標驗證:

- 30 秒視窗測試準確率：Mean Accuracy 約 81.52% ± 19.75%
- 校正後樣本準確率：Sample Accuracy 約 80.49%
- 推論延遲：自資料收集完成至輸出結果約小於 2 秒

圖 3.4. 1.5 End-to-End 完整作業流程圖

3.5 輸入/輸出設計說明

本節說明本系統於感測端、推論端與使用者介面端之輸入與輸出資料設計，作為系統資料交換、畫面呈現與前後端整合之依據。輸入/輸出設計分為共用輸入資料、共用輸出資料，以及消費端 App 與農民端 App 之額外輸入/輸出內容。

3.5.1 共用輸入設計

本系統之共用輸入資料主要包括下列項目：

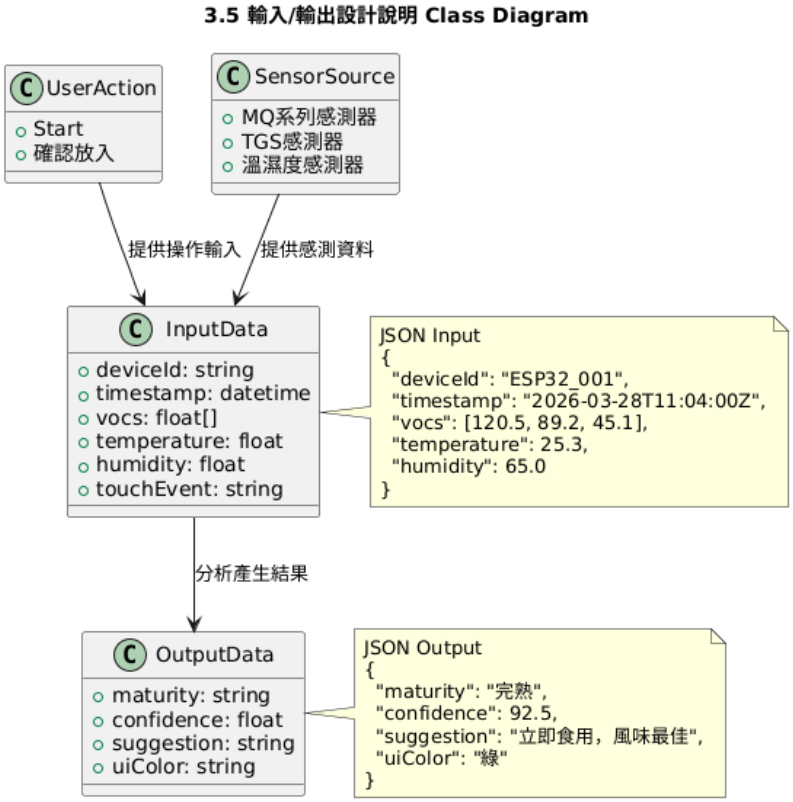
輸入內容:

- 感測器資料：MQ 系列與 TGS 系列氣體感測器之 VOC 數據，以及 BME280 之溫度、濕度、氣壓資料（JSON 格式）。
- 使用者操作：例如開始掃描、確認放入、語言切換、批次建立等觸控事件。
- 時間/裝置資訊：掃描時間戳、裝置 ID。

3.5.2 共用輸出設計

本系統之共用輸出資料主要包括下列項目：

- 輸出內容:
- 熟度分類輸出：4 類成熟度結果（未熟、初熟、完熟、過熟）。
- 信心值輸出：以百分比表示模型對預測結果之信心程度。
- 建議：文字如「立即食用」或「冷藏保存」。
- UI 顯示：顏色標籤（綠/黃/橙/紅）。



3.5. 1.1 輸入/輸出設計說明 Class Diagram

3.5.3 消費端 App 輸入/輸出設計

消費端 App 主要支援一般消費者進行單顆鳳梨掃描與結果查詢，其輸入/輸出設計如下：

輸入：

- 使用者點擊開始掃描
- 使用者確認已放入鳳梨
- 使用者選擇語言
- 使用者查詢歷史紀錄或知識內容

輸出：

- 單筆熟度辨識結果
- 信心值

- 食用建議
- 歷史查詢結果
- 知識內容頁面顯示

3.5.4 農民端 App 輸入/輸出設計

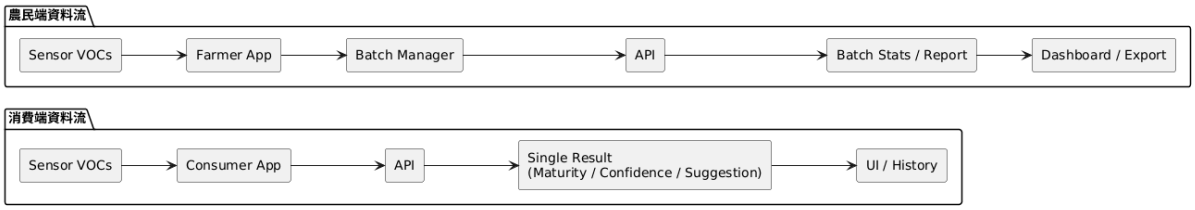
農民端 App 主要支援批次管理、批次掃描與統計分析，其輸入/輸出設計如下：

輸入：

- 批次名稱、數量、日期等批次建立資訊
- 批次掃描操作
- 報表查詢條件
- 歷史批次篩選條件

輸出：

- 批次掃描結果
- 批次熟度分布
- 統計摘要
- 報表分析結果
- 批次歷史紀錄



3.5. 1.2 輸入/輸出流程圖（Input/Output Flow Diagram）

3.6 其他設計說明

本節補充說明本系統於 App 端之操作流程、狀態轉換、錯誤處理、提示設計與後續擴充方向，作為系統操作邏輯與使用者體驗設計之依據。

3.6.1 系統操作流程（User Flow）

本系統依使用者端別不同，分為消費端與農民端兩類操作流程。

消費端流程著重於單顆鳳梨掃描、結果顯示與知識／歷史瀏覽；農民端流程則著重於批次建立、批次掃描、結果彙整與報表管理。

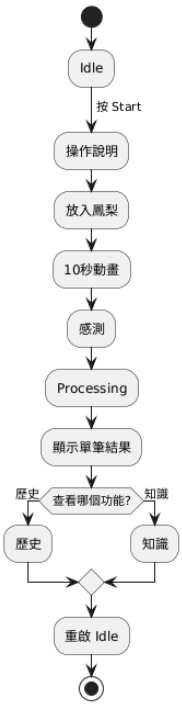
1. 消費端 User Flow

- 使用者自首頁進入系統
- 點擊開始掃描
- 依提示放入鳳梨

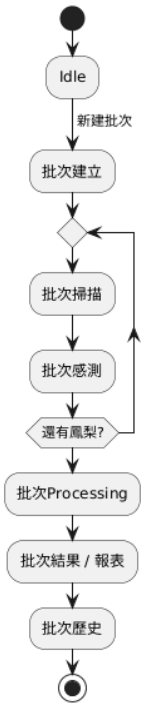
- 系統進行掃描與處理
- 顯示熟度結果
- 使用者可進一步查看歷史紀錄或知識內容

2. 農民端 User Flow

- 使用者自首頁進入農民端系統
- 建立新批次
- 執行批次掃描與感測
- 系統進行批次處理
- 顯示批次結果與報表
- 使用者可進一步查詢批次歷史



3.6. 1.1 消費端活動圖（Activity Diagram）- 消費端 User Flow



3.6. 1.2 農民端活動圖（Activity Diagram） - 農民端 User Flow

3.6.2 系統狀態機（State Machine）

本系統介面會依操作進度切換不同狀態，以反映當前任務流程， 消費端與農民端之狀態設計如下。

(1) 消費端狀態機

消費端狀態包含：

- Idle
- Instruction
- Scanning
- Processing
- Result
- History

其狀態轉換邏輯為：使用者由 Idle 狀態進入操作說明（Instruction），完成放入後進入 Scanning，掃描完成後進入 Processing，接收結果後進入 Result，必要時可再進入 History 查詢歷史紀錄。

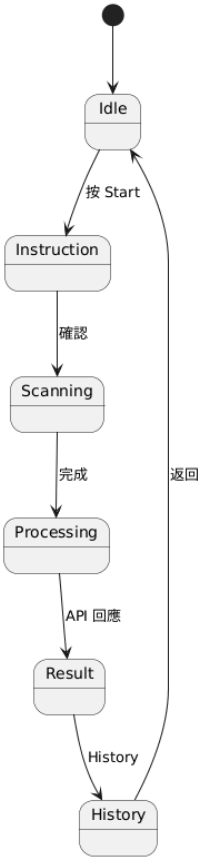


圖 3.6. 1.3 消費端狀態圖

(2) 農民端狀態機

農民端狀態包含：

- Idle
- BatchCreate
- BatchScanning
- BatchProcessing
- BatchResult
- Report

其狀態轉換邏輯為：使用者由 Idle 狀態建立批次（BatchCreate），進入批次掃描（BatchScanning）後，系統進行批次處理（BatchProcessing），再輸出批次結果（BatchResult）與報表（Report）。

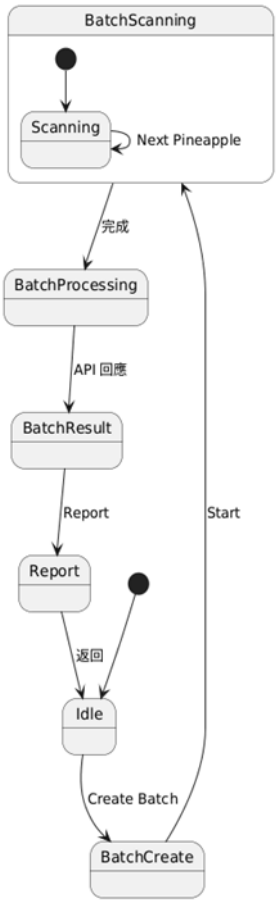


圖 3.6. 1.4 農民端狀態圖（State Diagram）

3.6.3 錯誤處理設計

本系統為提升操作穩定性與使用者可理解性，針對常見異常狀況設計對應之錯誤提示與回復機制。

1. 消費端錯誤處理
   - 感測失敗：顯示「感測器錯誤，請檢查連接」並返回待命狀態
   - API 錯誤：顯示錯誤訊息並提供重新操作或重試按鈕
   - 無資料：提示「無有效 VOC 數據，請重新放入」
2. 農民端錯誤處理
   - 批次建立失敗：提示批次資訊未完整輸入
   - 批次資料不完整：提示重新掃描或補齊資料
   - 上傳中斷：提示網路異常並保留未完成批次
   - 報表載入失敗：提示重新整理或重新匯出
3. 通用錯誤處理原則
   - 提示文字應簡潔清楚，避免過度技術化
   - 發生錯誤時應保留必要操作紀錄，以利回溯

- 介面應提供返回、重試或重新掃描等明確選項

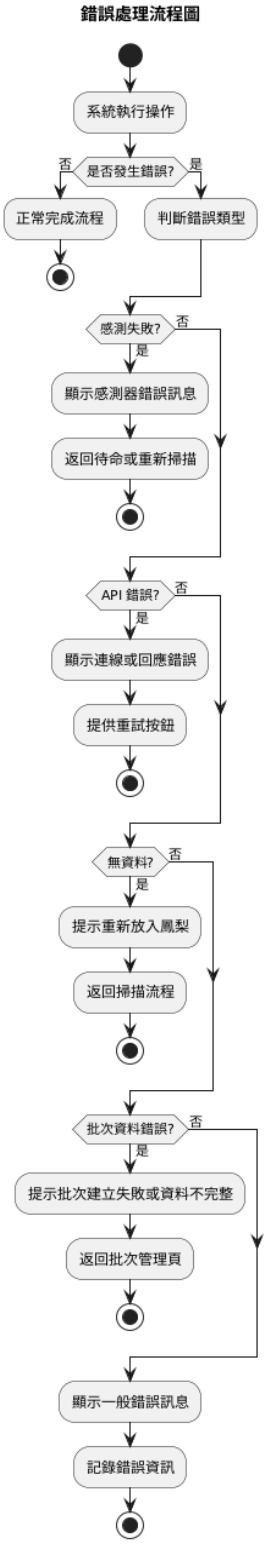


圖 3.6. 1.5 錯誤處理流程圖

3.6.4 音效與語音提示設計

為提升使用者操作流暢性與互動體驗，本系統提供音效與語音提示功能，並支援中英雙語。

（1）消費端提示設計

消費端以完整語音引導為主，協助一般使用者完成掃描操作，例如：

- 開始提示

English: “Please place the pineapple into the sensor. When ready, tap Ready to Scan.”

Chinese:「請將鳳梨放入感測裝置中，放好後請點擊放入完畢。」

- 點擊開始提示

English: “Great. I will start scanning in ten seconds.”

Chinese:「好的，十秒後開始掃描。」

- 開始掃描提示

English: “Scanning now. Please keep the pineapple steady.”

Chinese:「開始掃描，請保持鳳梨穩定。」

（2）農民端提示設計

農民端以簡潔提示音與狀態提示為主，避免在批次操作情境下造成干擾，設計重點為快速辨識當前狀態與批次作業是否完成。

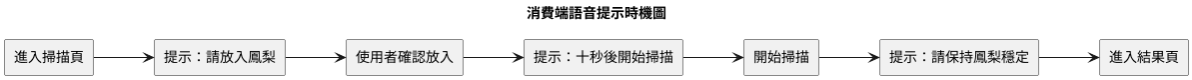


圖 3.6. 1.6 消費端語音提示時機圖

3.6.5 可擴充性設計

本系統於介面與功能設計上保留後續擴充空間，主要方向如下：

- 多種鳳梨支援：未來可透過模組化 API 與資料結構擴充不同品種鳳梨之熟度檢測功能
- 教育內容擴充：消費端可增加更多水果知識、保存方式與互動式內容
- 批次分析擴充：農民端可增加更多批次統計、報表分析與歷史比較功能
- 多語系支援：除中英雙語外，後續可再加入更多語系
- 多模態整合：未來可結合影像辨識模組，形成氣味與影像之整合判讀系統
- 充更多教育互動與水果知識模組；農民端可擴充更多批次分析、報表與多水果管理功能。

4. 需求追溯與版本管理

本章說明本系統於需求追溯、版本管理與附錄資料整理之方式，作為後續程式維護、文件修訂、功能擴充與成果追蹤之依據。由於本系統同時包含感測端、模型推論端、Flask 模擬介面，以及消費端與農民端 App，因此需透過一致之版本管理與檔案對照方式，確保各模組設計與實作內容可相互對應並具備可追溯性。

4.1 需求追溯說明

本系統之需求追溯範圍涵蓋硬體感測模組、模型推論模組、Flask 模擬介面、消費端 App 與農民端 App。各項需求應可回溯至本規格書相關章節、介面模組與實作檔案，以利開發、測試與後續維護。

需求追溯原則如下：

- 系統需求須可對應至設計規格書中之相關章節。
- 各功能模組須可對應至實作檔案與頁面模組。
- 重要輸入／輸出資料須可回溯至 3.5 輸入/輸出設計說明。
- 使用者流程、狀態機與錯誤處理設計須可回溯至 3.6 其他設計說明。
- 若功能需求、UI 畫面或模型邏輯有變更，須同步更新本規格書內容。

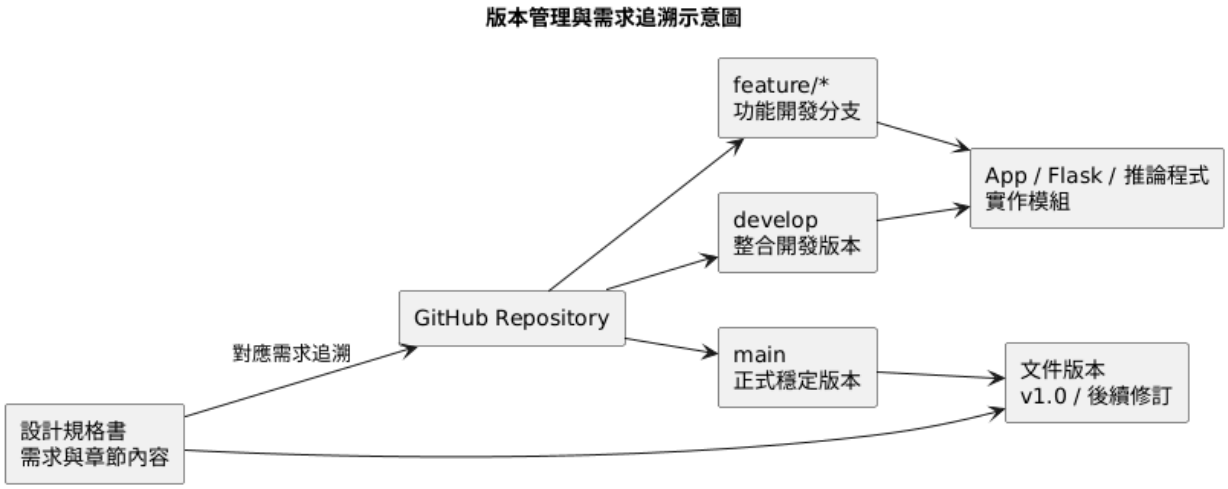


圖 4.1. 1.1 版本管理與需求追朔圖

4.2 程式版本管理

本系統程式碼採 GitHub Repository 進行版本控制，以支援多人協作開發、功能修改與歷史紀錄追蹤。版本管理範圍包括 Arduino 韌體、Raspberry Pi 推論程式、Flask 模擬介面程式，以及消費端與農民端 App 程式。

版本管理方式如下：

- 版本控制平台：GitHub Repository
- 主要管理內容：Arduino 韌體、calibrate_air_30s.py、inference_30s.py、applocal.py、React Native App 程式與相關設定檔
- 管理目的：記錄功能修改歷程、降低多人協作衝突、支援版本比較與回溯

本節說明消費端與農民端 App 之 React Native 檔案架構與程式模組對應關係，作為頁面實作與需求追溯之補充依據。

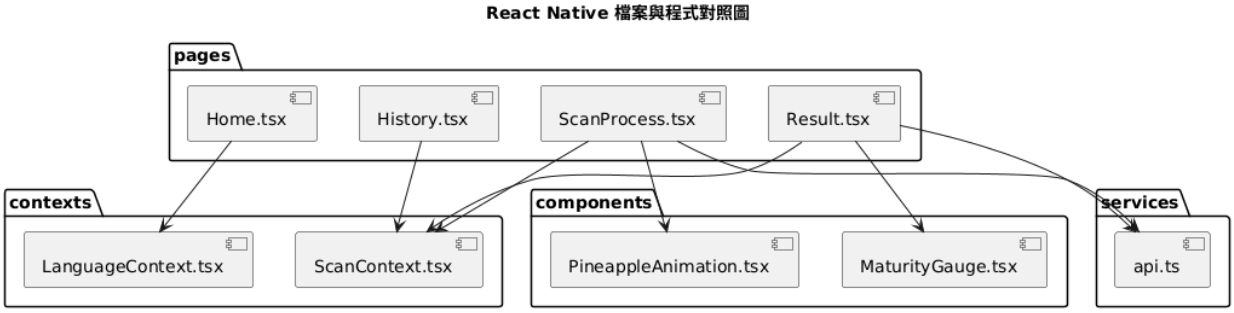


圖 4.2. 1.2 React Native 檔案與程式對照圖

4.3 分支策略

本系統採以下分支策略進行開發管理：

- main：正式穩定版本，保存可展示或可交付之版本內容
- develop：開發整合版本，供各功能完成後進行整體測試
- feature/*：功能開發分支，供個別模組、頁面或功能獨立開發使用

透過上述分支策略，可將穩定版本、整合測試版本與功能開發版本分開管理，以提升多人協作效率並降低版本混淆風險。

4.4 API 與系統版本規劃

本系統後端與資料交換介面可依功能擴充需求進行版本管理，初步規劃如下：

- v1：鳳梨成熟度四階段分類功能
- v2：預留未來多水果支援與功能擴充版本

此版本規劃可作為後續擴充多水果辨識、批次分析、報表管理與更多資料交換格式時之基礎。

4.5 文件版本控管

本設計規格書採版本化方式維護，目前文件版本為 v1.0。後續若有功能新增、模型更新、頁面調整或需求變更，應同步修訂本文件內容，並保留版本變更紀錄，以利後續追蹤與回溯。

文件版本控管原則如下：

- 功能變更需同步更新設計規格書。
- 文件修訂須記錄版本號與修改內容。
- 圖表、流程圖、檔案對照表若有更新，亦須同步維護。
- 文件版本資訊可搭配 GitHub Wiki 或專案版本紀錄進行管理。

附錄

附錄收錄本系統之 UI 訊息清單、檔案與程式對照表，以及檔案與報表對照表，作為本設計規格書之補充參考資料，並用以支援需求追溯與實作對照。

(1) 訊息清單

本節彙整系統介面中常用之顯示訊息與雙語文字，供消費端 App、農民端 App 及相關提示設計參考。

UI 顯示文字（中英）

- Start Scan / 開始掃描
- Insert Pineapple / 放入鳳梨
- Processing... / 處理中...
- Maturity: Ripe / 熟度：完熟
- Confidence: 92% / 信心值：92%
- Suggestion: Eat now / 建議：立即食用

(2) 檔案與程式對照表

本節說明主要程式檔案與其對應功能模組，作為程式實作與設計規格之對照依據。

A. 消費端 App

檔案	對應報表/功能
Home.tsx	3.3.2 A 首頁模組
ScanProcess.tsx	3.3.2 A 掃描流程頁
Result.tsx	3.3.2 A 結果頁
History.tsx	3.3.2 A 歷史紀錄模組

B. 農民端 App

檔案	對應報表/功能
FarmerHome.tsx	3.3.2 B 農民首頁模組
BatchCreate.tsx	3.3.2 B 批次建立模組
BatchScan.tsx	3.3.2 B 批次掃描模組
BatchResult.tsx	3.3.2 B 批次結果模組
ReportAnalysis.tsx	3.3.2 B 報表分析模組

C. 系統與模擬介面程式

檔案	對應功能
calibrate_air_30s.py	3.2.2 基準線校正模組、3.4.2 系統校正作業流程
inference_30s.py	3.2.2 推論引擎模組、3.4.3 成熟度檢測作業流程
applocal.py	3.2.2 Flask 模擬介面模組、3.3.5 Flask 模擬介面與驗證流程

(3) 檔案與報表對照表

本節說明系統檔案、畫面模組與報表輸出之對應關係，供後續測試、維護與成果整理使用。

檔案	對應報表/功能
Home.tsx	首頁與語言切換
ScanProcess.tsx	單筆掃描流程與量測引導
Result.tsx	單筆熟度結果、信心值與食用建議
History.tsx	歷史查詢與紀錄顯示

B. 農民端 App

檔案	對應報表/功能
FarmerHome.tsx	批次管理首頁
BatchCreate.tsx	批次建立
BatchScan.tsx	批次掃描流程
BatchResult.tsx	批次結果彙整
ReportAnalysis.tsx	統計分析與報表匯出